HOLY SPIRIT UNIVERSITY OF KASLIK

Faculty of Business and Commercial Sciences

Financial Engineering Division

Developing Successful Algorithmic Trading Systems

Financial Engineering MBA Thesis

Prepared by

Bernard Fares

Under the direction of

Dr. Charbel Salloum

Kaslik-2012

“Everything should be made as simple as possible, but not simpler.”

–Albert Einstein

This thesis is dedicated to my mother and father.

If I gather up all the money in this world I wouldn’t be able to repay you.

Acknowledgements

First and foremost, I would like to thank “the King” Andre Eid for his great teachings and support all through these years. He taught me more than finance; he taught me how to think and has really been a great inspiration for me.

A special thank you goes to the director of my thesis, Dr. Charbel Salloum, for his continuous help and supervision. It is such an honor to work with a person who achieved so much in his life and yet remained modest enough to sacrifice his time to help students.

I would also like to thank the Holy-Spirit University for backing me in my journey and giving me the opportunity to pursue my goal of becoming a financial engineer.

Contents

Chapter 1 8
Algorithmic Trading 8
Abstract 8
Introduction 8
Algorithms in the Trading Cycle 10
Algorithmic Strategies 11
A. Trend Following 11
B. Pairs Trading 12
C. Delta Neutral Strategies 12
D. Arbitrage 12
E. Mean Reversion 12
High Frequency Trading 13
Trade Cost Reduction 13
A. Iceberging 14
B. Pegging 14
C. Smart Order Routing 14
D. Simple Time Slicing 15
Developing Algorithmic Trading Systems 15
A. Electronic Exchange Platform Development 15
B. Algorithm Development 16
Stage I. Research and calculations 16
Stage II. Back-test 17
Stage III. Implementation 18
Stage IV. Portfolio and risk management 19
Chapter 2 21
Introducing the Algorithm Components 21
The Trading System Overview 21
Trend Following 22
Trend Following Strategies 23
A. Simple Moving Average Crossovers 23
B. Channel Breakout 24
Artificial Neural Networks 26
A. Simple Neuron Structure 26

B. Transfer Functions 27
C. Training Neural Networks 28
D. Neural Network Structure 29
E. Normalizing the Data 31
F. A Financial Example 32
The Average Directional Index (ADX) 34
A. Introduction 34
B. Interpretation 35
C. DI Crossover Strategy 35
Stochastic oscillator 35
William %R 36
Value at Risk (VaR) 36
A. VaR Calculation 36
1. Historical Simulation 36
2. Model Building Approach 37
Chapter 3 39
The Implementation 39
The Electronic Exchange Platform 39
The Strategy 42
Modeling the Algorithm 42
The Coding 45
Implementing the ANN 48
A. Why Choosing ENCOG? 48
B. The ANN Process 48
C. Creating the Inputs 49
D. The Neural Network Structure 49
E. Training The ANN 49
F. Using the Trained Neural Network 50
Chapter 4 51
The Empirical Study 51
Generalization vs. Optimization 51
The Drawdown 52
Metatrader 5's Strategy Tester 52
Training the Network 52

The Tests 53
A. Preliminary Tests 55
B. Finding the Best Parameters 62
C. The Final Back-test 65
Running the System 67
Conclusion 68
Bibliography 69

Chapter 1

Algorithmic Trading

Abstract

Electronic trading became a widely used trading process. The use of electronic platforms to execute trade orders and to apply money management techniques dominates various financial markets.

This study investigates algorithmic trading. It highlights its pros and cons, concentrates on algorithmic trading powerfulness, and shows how to execute trades effectively and efficiently in today's highly volatile markets.

During this study, we will show how to develop algorithmic trading systems by developing our own automated trading and money management system. The system's underlying trading strategy will use both neural networks and trend following techniques to execute trading orders. Risk measurement techniques will also be integrated. After building and implementing the system, we will show how to maintain it and measure its performance.

This study will be based on the foreign exchange market; several pairs of currencies will be used to back-test, generalize the system's parameters, and analyze its performance. Finally, an empirical study will take place.

Introduction

Algorithmic trading, also called algo-trading or automated trading, is the use of electronic platforms that, with the help of mathematical models and a set of rules, analyze every market quote and trade, then turn the results into a trading decision. In many cases, the orders are initiated without human intervention. Algorithmic trading is a way to codify the trader's execution strategy, and cut down and improve transactions costs. It also helps investors forecast market data that the speed of financial markets' information made a complex task nowadays.

Each algorithmic trading system is driven by a programmed script that contains a set of rules, methods and functions used to perform a certain task. This script is referred to as an algorithm. In its initial stages, algorithmic trading started with first generation

algorithms that were just pure execution algorithms, simple with their goals and logic, before it evolved to second generation algorithms. These were more sophisticated, capable of implementing strategies and providing signals that were executed by trade execution algorithms. Later came the third generation algorithms, or smart algorithms, that were much more advanced, as they contained intelligent logics that learn from the market activity and adjust the trading strategy to suit the market conditions.

Over the last decade, algorithmic trading caught the attention and became the focus of large financial institutions, securities regulator, and different exchanges, and it replaced dramatically most of the traditional trading systems; even brokers can no longer fulfill order flow without using some method of electronic execution. But what should be mentioned is that algo-trading isn't new, it's almost as old as computers themselves.

Algorithmic trading started in 1951 when Harry Markowitz, a Chicago University student, developed the modern portfolio theory. In 1960, with the help of "IBM System/360", a system using this technology was built based on Markowitz's theory. The system analyzes millions of data points generated by the markets.

These techniques were put to use in the 70's and 80's by investment houses to guide trades.

As for the orders' flow, it was also computerized in those 2 decades, when the New York Stock Exchange introduced respectively "Designated Order Turnaround" system (DOT) in 1976, and later Super-DOT in 1984. These two systems allowed the transmission of orders to buy and sell securities to the proper trading post electronically. A special electronic workstation called the "display book" displays the orders and allows each specialist firm to execute orders for the market. A new financial era started!

Previously, in the floor-based trading, buyers and sellers stood literally in front of each other to accomplish a deal. However, floor-based trading drew to a close in the 1980s, where fully electronic markets dominated and made executing orders much faster, which allowed algorithmic trading to spread widely.

Algorithms in the Trading Cycle

The utilization of algorithmic trading alone does not guaranty better results; the traders should be more aware of how to choose the best algorithmic strategies that meet their investment objectives.

Before selecting the strategy, a pre-trade analysis should be performed to assess if the order is suitable for an algorithmic trade. When the order is deemed, the trader should specify a benchmark price and implement his goals (aggressive or passive execution usually specified in the algorithm). He should also specify deviation rules that tell the algorithm how to behave when the market movement changes. Finally, a post-trade analysis is essential for the trader to measure the algorithm performance and ensure its consistency with the overall investment objectives.

A trade cycle has three different stages in which algorithms can be implemented. They are classified as following:

- Pre-trade analytics
- Execution stage
- Post-trade analytics.

Pre-trade analytics can be defined as the analysis of potential trade opportunities by using the market data which will be analyzed in the purpose of providing the necessary output to make an informed algorithmic trading decision.

The trader is provided with liquidity summaries, cost and risk estimates, summary of all the difficulties and stability measures that determine the orders suited for an algorithmic trade. He can also be notified on hedging and risk reduction opportunities to improve his trade.

Execution stage is the phase where, in order to enter the market, the implemented algorithm puts the strategy in action or chooses an appropriate strategy, among several, that suits the pre-trade outputs. In this stage, algorithms and their parameters are monitored for their performance and are subject to change.

Post-trade analytics is a two-part process that consists of execution cost measurement and algorithm performance analysis. Execution costs are the difference in value between a benchmark trade and the actual trade's final average price, including commissions, fees and all other costs. As for algorithmic performance, it is analyzed in the purpose of assessing the algorithm's ability to stick to the optimally prescribed

strategy, to achieve reasonable prices and to analyze its behavior in applying deviation strategies when markets deviate.

Algorithmic Strategies

One of the advantages of algorithmic trading is the ability to put most of the traditional trading techniques and strategies that were used in the old fashioned way, into a well-structured programming algorithm so that they run electronically in a more efficient and effective way.

As for new strategies, the fact that all the financial indicators are computed dynamically on the spot with each new quote's tick, and statistical methods are applied within the algorithm to help with the trading decision, has given traders and investors the ability to create any custom strategy, using modernized more advanced and fast techniques. These techniques can help investors and traders analyze the market in a flash to predict it or follow it, in the aim of placing successful orders.

What should be mentioned and better discussed is the importance of controlling, updating and maintaining algo-trading systems and their underlying strategies.

Many well maintained strategies proved to be performant under an algorithmic trading environment. We will shortly discuss the most common ones.

A. Trend Following

Trend following is an old technique used by many successful traders, such as the legendary commodity trader Richard Dennis who turned an initial stake of $ 5,000 into more than $ 100 million, and taught his famous "turtle traders" this same technique that made them successful.

As its name indicates, this strategy consists of spotting trends and following them. That means going long when it's an upward trend, conversely, going short in case of downward trends. There are plenty of indicators and methods that can be used to detect and spot trends, they provide entry as well as exit signals. The most popular ones are:

- The channel breakout method
- The double crossover method
- The ADX indicator.

B. Pairs Trading

Pairs trading is a market neutral strategy. It consists of taking two antagonistic positions by short selling a security, and buying another one in a predetermined ratio. The resulting quantity is called spread; it is computed using the two securities' prices and it forms a time series. Orders are executed when the spread is away from its mean value and expected to revert back.

The principal idea behind pairs trading is that securities with similar characteristics must be priced more or less the same. The spread is considered as the mispricing. The strategy assumes taking long-short positions when the spread is away from its mean, and this is done with the expectation that the mispricing is likely to correct itself.

C. Delta Neutral Strategies

Delta neutral strategy is defined as keeping a portfolio's value neutral to small changes in the underlying. Such a portfolio typically contains options and their corresponding underlying securities. The hedging can be established by buying or selling an amount of the underlier that corresponds to the delta of the portfolio. The amount, bought or sold, will sum the portfolio's delta to zero.

D. Arbitrage

Arbitrage is defined as the simultaneous purchase of an asset from a particular market and selling it in another one, in order to profit from the price difference. Arbitrage is the result of market inefficiencies, and it is considered a mechanism to ensure that prices don't deviate substantially for long periods of time from their fair values.

E. Mean Reversion

This strategy considers that securities tend to have an average price over a time period. When the security price is below average, investors consider the security underpriced and subject to purchase, with expectations that its market price will revert back to average. As for a security which price is above average, it is considered overpriced and suitable for sell.

High Frequency Trading

High frequency trading is a subset of algorithmic trading where large numbers of orders are executed at high speed, with network latencies measured in micro seconds. Usually, orders taken by high frequency trading systems are small, and have a small life span measured in seconds. These orders are conducted on high speed computers, running programs with sophisticated algorithms that analyze market data and provide trading signals in milliseconds. High frequency trading is not a strategy; it is rather a new technique with advanced methods of implementing new strategies that benefit from market liquidity imbalances or other short-term pricing inefficiencies.

Trade Cost Reduction

An optimal trading strategy should begin with the measurement of the trading costs and implementation shortfall. Implementation shortfall can be defined as the difference in return between a theoretical portfolio (or trade) and the implemented portfolio (or trade). It measures the distance between the realized price and a pre-trade decision price. Before executing any trade, a forecast of the trade's cost must be envisioned and used in the post trade to measure performance.

Trading costs are measured by looking at several factors, such as the bid and ask price at which an investor can purchase or sell an asset. This price can differ from the theoretical portfolio prices. Another factor is the price impacts with liquidity, where the bid-ask prices are impacted by large trades, and are changed due to lack of liquidity. Other factors such as commissions, fees and taxes are also known as explicit costs, which are unavoidable costs and can significantly affect a portfolio. Taxes are important because some strategies expose investors to much greater tax liabilities than others. A strategy in which there is a high frequency of trades may be affected by higher taxes than a strategy depending on long time horizon trades.

The rise of technology and the development of sophisticated systems have changed the way traders access the market. New order routing technologies and algorithmic strategies shifted the responsibility of best execution, from traditional brokers to money managers themselves.

The increasing use of electronic trading systems should be accompanied by specific benchmarks that make the investor capable of measuring his trades' quality and

performance. The computation of the benchmark became an easy task thanks to the accessibility of electronic platforms to the clients.

Let us take for example the Volume Weighted Average Price or VWAP, a popular stock trading benchmark. To compute the VWAP, we simply need to access the daily stocks transactions records, and add up the dollar value of each transaction (price times shares traded), then divide by the total shares. Using VWAP to judge shares is fairly simple. A buy trade with price below the VWAP indicates a good trade, while a price above the VWAP indicates a bad one. For a sell trade, the valuation is reversed. The use of new order routing techniques and trade benchmarking, such as VWAP, is steadily dropping transactions cost and forcing brokers-dealers to use computers and automation to cut costs. Buy side traders are concentrating more on cost research and on choosing the proper algorithm integrated with an order management system.

Trading costs can also be improved by minimizing the market impacts and achieving the best executions which can be done by algorithmic strategies. The strategies discussed shortly below are the most common ones.

A. Iceberging

Iceberging is a technique used to hide large orders from the market participants to prevent high price impacts. Using automated mechanisms, a large order is dissected into smaller ones to hide the actual order size. The participants only see a small portion of the total order; it is similar to an iceberg where only the tip of an immense mass of ice shows.

B. Pegging

A pegged order is a limit order to buy or sell a security at a price set to track the bid-ask prices, which mean that if the bid-ask price moves, the order is modified accordingly. Each pegged order has an entry price and a limit price which, if reached, will cause the pegging to stop.

C. Smart Order Routing

Smart order routing routes orders to leading exchanges that have the best liquidity. The investors benefit from extra liquidity by reducing cost and choosing the utmost

execution price. This strategy is conducted using complex event processing applications that compare each exchange's liquidity and choose the best one.

D. Simple Time Slicing

Simple time slicing splits up market orders and send them for execution at a regular time interval.

Developing Algorithmic Trading Systems

Developing trading systems encompasses two parts. The first part is building or developing an electronic exchange platform that communicates with the exchanges, in order to get market data and place orders. The second is building the underlying algorithm upon which all the automated trading will be driven.

A. Electronic Exchange Platform Development

The development of an electronic exchange platform starts by choosing a programming environment and an appropriate programming language (such as C++.NET). Using Application Programming Interfaces (APIs), we link the platform to the market over which the members of exchange trading firms can access exchange data and place orders electronically. The development of this program involves installing and maintaining the network's infra-structure responsible of communications and orders execution. This process deals with complex networks, servers, routers and FIX engines (protocol defining the messaging specifications for the electronic communication of trade-related messages). Rather than spending time, effort and expenses to build such an electronic exchange connections from scratch, we can use third party trading software connections and built-in functionalities such as data feeds, order entry and risk management. Due to the fact that this study concentrates more on developing trading algorithm strategies, we are not going to build an electronic exchange platform, instead we will be using the Metatrader5's exchange connection and functionalities. Further details will be discussed later on.

B. Algorithm Development

As mentioned earlier, the trading algorithm is the underlying script that controls when, where and how the platform executes orders. The algorithm can cover the whole trading cycle functions from the pre-trade analytics, to the post-trade analytics. It's at this scope that the market data is analyzed, interpreted and converted to executable orders, depending on the underlying strategy.

Four stages are involved in creating quantitatively based algorithms.

Stage I. Research and calculations

Each successful trading system starts by a plan that gives its developer a clear idea on how to start and where to head. Complex systems that offer long term opportunities are planned and built step by step and they evolve consecutively with new ideas.

1. Describe the trading technique

This first step serves as an idea about the trading system's logic and its quantitative methods. The more complex the idea is, the more difficult it becomes to communicate it. A documented description of this idea would serve as a great starting point.

2. Research quantitative methods

Thinking of new ideas and new methods became rare. Usually we prefer building old systems with a twist, which put us in a situation where we find ourselves researching mathematical models. As a result, either we use publicly available models and techniques, or we add up a certain kind of innovation. This act of research can turn out very beneficial, as it gives us some understanding on the most successful methodologies. The goal of the research is to refine our methods and techniques in the purpose of building a successive trading algorithm.

3. Creating a prototype

Developing the trading technique with any Modeling software (Excel, Matlab, Eviews…) is a great way to test it. Prototyping helps create several scenarios to test whether the technique is consistent or needs further investigation.

Trading techniques in which we have high confidence to make profits are often built immediately into a trading system; the problem with such systems is that they are not well defined. Prototyping helps defining algorithms and data requirements.

4. Performance check

Not all systems can be tested using modeling software. If we find ourselves unable to test the system, we cannot proceed to the following stages. Furthermore, if at any point in this stage or the upcoming ones the performance testing shows a system failure, then algorithm development should be stopped, or we should loop back to previous stages. The goal is to suspend the development of trading systems with low probability of success.

Stage II. Back-test

Successful systems are validated and analyzed using past market movements; this process is called back-testing. Back-testing is the process of testing a trading system using historical data, before performing statistical analysis to the results.

1. Gather historical data

After passing the first stage and proving that the system is worth further investment of time and resources, the back testing should begin. Before building and implementing the system, it must be tested manually on a large historical data, preferably for several instruments.

2. Data filtering algorithms

The integrity of a system is enormously linked to the integrity of its data. The purification of data is essential; it can be done by developing a Data Transformation Management System (DTMS) that scans data for errors and irregularities, and correct them. The DTMS should clean both live and historical data.

3. In-sample and out-of-sample test

The in-sample and out-of-sample test is probably the most critical step in this process.

The results between in-sample and out-of-simple may differ significantly due to several facts, like optimizing the system's parameters to the historical data, or due to unconsidered change in the markets' behavior. Such tests result in one of the three following outcomes:

- Profitable test results both in-sample and out-of-sample
- Profitable test results in-sample, but not out-of-sample
- Unprofitable both in-sample and out-of-sample.

If the system is performant, in both in-sample and out-of-sample, then we can proceed to implementation. If the system shows acceptable performance only in the in-sample test, then we should loop back to previous stages and try out more researches and changes. However, if the system proves to have unsatisfactory performance in both in-sample and out-of-simple, then it is likely the system should be rejected.

Stage III. Implementation

Building an automated trading system (fully or partially automated) is a software development project. Implementation needs proper hardware, a programming environment such as visual studio, a programming language and good programming skills.

1. Software specification document

The purpose of this document is to fully define the functionalities and performance requirements of the trading system. This document will simplify the programming procedure and help the developer quickly implement the correct functionalities and methods. This document should clearly define the steps of the project along with a documentation of all the detailed information about the trading algorithm. Fortunately, much of this work has been already done in the previous stages.

2. Document and design the system architecture

A system architecture document should be formulated containing detailed information about the hardware and software that form the architecture of the trading system. The document includes financial calculations, real-time data, user interfaces, order routing connections and any other necessary processes.

The system architecture document is considered as the roadmap for coding the algorithm. The more complex the trading system becomes, the more detailed the system's architecture document and the architectural plans should be. A Unified Modeling Language (UML) should be used to compose this document and to describe the system's architectural plans.

3. Programming and documenting the system

Once the system's architecture document is ready, the programming process starts.

The programming procedure will be a step by step march through the software's architecture document. It should be mentioned that the process of programming includes proper software testing.

Finally, once the programming finished and the software well tested, a user manual should be produced. The manual helps traders operate the trading system, and programmers maintain it.

4. Probationary trading

After building the trading system, we put it into a probationary trading period with small sized orders. The main purpose of probationary trading is to discover errors and flaws in the trading algorithm before investing the full sum. The second purpose is to make traders familiarize themselves with the trading system, and to determine whether building any additional tools is needed to properly manage the market risk produced by the trading system.

Stage IV. Portfolio and risk management

It's not enough to build and implement a system, successful systems need continuous monitoring. The monitoring involves generating periodic reports that show the system's performance. The reports should contain trade performance statistics, risk statistics and the systems P&L. Furthermore, when the trading system deviates from the expected results, the reports should present the causes of variation and an action plan to deal with them.

1. Monitor portfolio statistics

Continuous monitoring and reporting of portfolio statistics, trade limits and risk factors should always take place. These reports help understand if the system is working within the specifications or if it is deviating from them. In case of deviation, the system's parameters should be updated and reassessed.

2. Risk calculations

Risk calculations and reports give management an idea about the trading system's profits and losses, as well as drawdowns. These calculations will help us asses the system's performance relative to the market and other industries.

3. Documenting P&L

Monitoring the system successfully should be done by keeping a track of individual trades and their respective payoffs. These will be useful when evaluating the system against a certain benchmark.

4. Repeat the above stages for continuous improvement

The above steps should be considered as a never-ending loop. Each time we reach the end, we should start again from the beginning to improve our system by refining it and evolving it by creating new ideas.

The continuous improvement involves improvements in management, quantitative analysis, traders and the coding of the algorithm. Through research and entrepreneurial activity, many breakthrough ideas can be discovered, such as the creation of new trade selection algorithm and the implementation of new technologies for more efficient trade executions.

Chapter 2

Introducing the Algorithm Components

After defining algorithmic trading and how to develop its related systems, we will start by introducing the algorithmic trading system that will be developed in the rest of this study.

The Trading System Overview

After several researches and calculations, the result was a hybrid trading strategy. By hybrid we mean a mix of old trading techniques and modern quantitative methods. A quick description of the trading system and its underlying algorithm will serve us as a good start to introduce the system's components and their functionalities, so that we could explain each one of them in detail.

Usually, traders or investors find themselves dealing with either a trendy or a sideways market. When markets are trendy they could either trend upward or downward. In both conditions, trend followers proved to be very performant and trend following testified to be profitable. For these specific reasons, trend following techniques will be used to deal with trade decisions when markets are trending. As for sideways market, it's not easy to trade in such conditions due to the uncertainties that will simply limit our opportunities. Applying trend following techniques in sideways markets could result in high losses or no profit at all. Sideways market can last for days, months or even years, so we cannot pause trading waiting for trends, thus we need to find a profitable method or technique that covers such markets. Using neural networks to forecast sideways market direction and to place orders based on this forecast can be a profitable strategy, due to the advanced forecast that neural networks provide. Since this quantitative method is only used in the sideways market, and the trend following technique in the trendy one, we need to implement indicators that detect these market conditions. There are plenty of indicators that help detecting trends; we will be using the most effective and efficient ones, detailing each one used in this trading system. Before executing any trade, VaR measurement methods are

integrated in the algorithm so that investors or traders could measure their risk and know in advance the probable value they can lose.

Trend Following

Many trading methods and techniques are continuously used and developed in the aim of making the highest profits and the best performance. These trading techniques and methods can be split into two categories, predictive or reactive.

The predictive category groups all the methods and techniques that use forecasting and market direction prediction as the basis of all orders execution.

In contrast, the reactive category groups methods and techniques that neither predict nor forecast the market, they just detect if the market is trendy and the direction to which it trends to.

Trend following is simply a strategy that uses reactive methods and techniques to detect the market's trend and respond to it by executing orders. This strategy deals with what has happened in the market rather than anticipating what will happen. Trend followers use statistical validated trading rules that don't get them emotionally involved and enable them to focus on the market.

Since trend followers follow the market movement and don't predict it, that makes them unable to buy the bottom and sell the top because the movement would have begun earlier. Trend followers don't execute orders frequently; they wait for the right entry signal at the right market conditions instead of forcing the market. As for closing positions, trend followers wait for an exit signal to step out of the market, which makes them unable to set performance goals or trading objectives.

The trend follower's discipline towards market adversity is a key element to achieving successful trades. These trades are based on an objective and automated set of rules, this way of trading is referred to as systematic trading. These rules are firmly followed and applied by traders to enter and exit markets. Eliminating human emotions and sticking to the rules are extremely important, and breaking them can result in great losses.

Since it's difficult to control human emotions and traders psychology, trend following was always fruitful to the extremely disciplined people, up until algorithmic trading arose. Algorithmic trading is considered the perfect solution to trend followers specifically and systematic traders generally. The trading rules are fed into an

algorithm, and the system will start executing the trades based on this algorithm without any human intervention. The only human intervention that takes place is when maintaining the system and changing its rules, or updating the rules' statistical parameters in case of performance deviation.

Trend Following Strategies

The two most commonly used trend following technical indicators are Channel Breakout and Simple Moving Average Crossovers. For the sake of our study we will do a fast preview on the Simple Moving Average Crossovers, and go more in depth with the Channel Breakout indicator that we are going to use in our trading system.

A. Simple Moving Average Crossovers

A Simple Moving Average or SMA is a lagging indicator that shows a security's average price over a period of time. The Simple Moving Average Crossovers is none other than two SMA having two different periods of time that cross each other, generating a trade signal. Let's take for example a 10x100 Simple Moving Average Crossover:

SMA10 = average of the last 10 closes

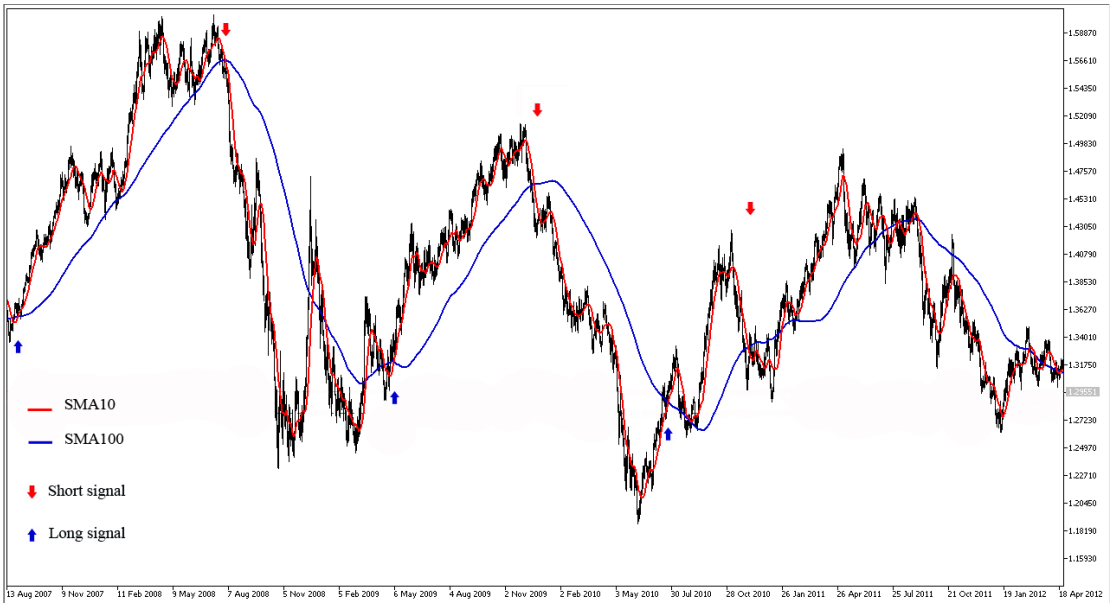
SMA100=average of the last 100 closes

The long signal is generated if SMA10 > SMA100.

The short signal is generated if SMA10 < SMA100.

The above signals can be considered as entry signals or exit signals in case there were opened positions. Figure 2.1 shows a daily EUR/USD chart from August 2007 till April 2012, with trading signals generated by a 10x100 SMA crossovers strategy, the red line being a 10 days SMA and the blue one being a 100 days SMA.

Figure 2.1: *Trading signals generated by SMA Crossover.*



B. Channel Breakout

Channel breakout or Donchian Channel Breakout is a famous leading indicator that gained its reputation from Richard Dennis and his turtle traders who made huge profits using it in the 1980s. The channel breakout is an asymmetric channel having the following rules:

If the market reaches the highest high of the X previous days, then we should enter long, and exit if it reaches the lowest low of the Y previous days with Y < X.

If the market reaches the lowest low of the X previous days, then we should enter short, and exit if it reaches the highest high of the Y previous days with Y < X.

Let's consider a 40 days channel breakout with a 20 days exit where:

C = close of the day

HC (40) = highest close in the last 40 days

LC (40) = lowest close in the last 40 days

HC (20) = highest close in the last 20 days

LC (20) = lowest close in the last 20 days

The long signal is provided and a buy position is opened if C > HC (40). This position is closed when C < LC (20).

The short signal is provided and a sell position is opened if C < LC (40). This position is closed when C > HC (20).

Figure 2.2 shows a typical turtle trading trade in November 1979 based on a 40 days channel breakout strategy that led to high profits.

Figure 2.2: *A Turtle traders' trade*

Source: Genesis Trade Navigator

- Choosing the right channel breakout parameters

Choosing the right channel breakout's entry and exit parameters is a very crucial and important decision that will affect the profitability of the whole trading system. Good parameters detect a trend being formed at an optimal time, neither early nor late. Small parameters detect trends earlier, but their probability of detecting false signals is high, more frequent and can negatively affect the profitability. As for large parameters, they can reduce the false signals probability and frequency, but they can also limit the profitability due to the fact that they detect the trend too late, which can cut off most of the profits.

There is no exact rule or method for choosing the channel breakout entry and exit parameters. Actually, each investor or trader chooses the parameters he finds more profitable, and which could detect a trend in its earliest stages and with minimal false signals. Parameters are chosen with a trial and error process conducted by back-testing the market's historical data and choosing the most performant parameters with

an acceptable drawdown. What's important to take into consideration is choosing generalized parameters that can adapt to the market's movements in the long run. These parameters are generalized in the process of choosing them by performing the trial and error on large historical samples that encompass several years of data. Parameters that are optimized to suit a small sample of historical data and are put into action for future trades can be useful for a short-term period of trading, but misleading on the long run trading horizon.

Artificial Neural Networks

Artificial Neural Networks or ANN are a small part of artificial intelligence. Inspired by how biological neural networks process information, ANN are mathematical constructs designed to process incoming data in the purpose of accomplishing small tasks. ANN are composed of simple elements that operate in parallel. The connection between these elements determines the ANN function.

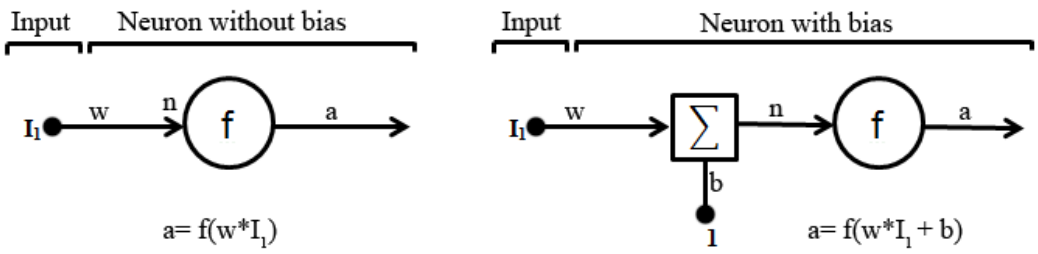
ANN are trained to perform a particular function; the purpose of the training is to adjust the values (weights) of the connection between elements so that a particular input leads to a particular output.

Neural networks can be used to accomplish many tasks such as:

- Data classification: input data is classified into groups.
- Prediction: future data is forecasted based on historical data.
- Pattern recognition: The neural networks recognize patterns in the incoming data.
- Optimization: Neural networks are used to optimize systems such as short-term trading systems.

A. Simple Neuron Structure

Figure 2.3: *Neuron structure*



In figure 2.3, the left neuron shows an input I_1 introduced to the network and multiplied by a certain weight w. The weighted input $n=I_1*w$ will be the parameter of the function f called transfer function, that will produce an output a. As for the right neuron, we added a bias b to the inputs before passing it to the transfer function.

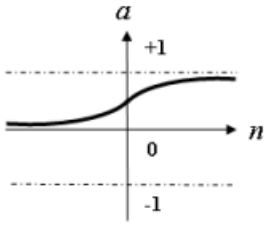
We should note that w and b are both adjustable parameters of the neuron. The basic idea of neural networks is that such parameters are adjusted to attend the desired behavior. The adjustment of these parameters is done during the training of the network.

B. Transfer Functions

The transfer function (also known as activation function) f shown in figure 2.3 can be a linear or nonlinear function of n. The transfer function is none other than a mathematical representation of the relation between the input and output. The transfer functions are usually one of the following functions.

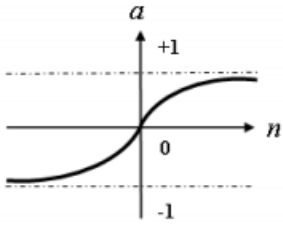
- Sigmoid function: $f(n) = a = \log sig(n) = \frac{1}{1+e^{-n}}$ (Figure 2.4) is one of the most commonly used transfer functions. This function can take any parameter between [-∞;+∞] as input and give an output between [0;1]. This function is used with multilayer networks that are trained using the backpropagation algorithm (will be discussed later).
- Hyperbolic tangent function: $f(n) = a = \mathrm{Tan}\, sig(n) = \frac{2}{1+e^{-2n}} - 1$ (Figure 2.5). This function can take any parameter between [-∞;+∞] as input and give a bipolar output in the range between [-1;1]. This function is used for neural networks where speed is more important than the exact shape of the transfer function.
- Purelin function: $f(n) = a = \mathrm{Purelin(n)}$ (Figure 2.6) is used to represent the input/output behavior for models that are linear or approximately linear.
- Hard-Limit function: $f(n) = a = \mathrm{hardlim(n)} = \{0 \text{ if } n < 0\ , 1 \text{ if } n \geq 0\}$ (Figure 2.7). The output of this function is binary, either 0 or 1. If n is less than 0, the output will be 0, but if n is bigger than 0, the output will be 1.
- Step function: $f(n) = a = \{0 \text{ if } n < \theta\ , 1 \text{ if } n \geq \theta\}$. The output of this function is binary, either 0 or 1, depending on whether the input meets a specified threshold θ.

Figure 2.4: *Sigmoid function*



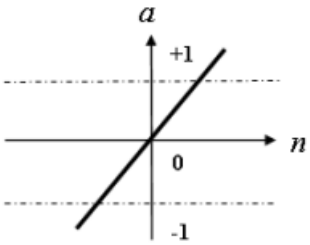
$a = \text{Logsig}(n) = \frac{1}{1+e^{-n}}$

Figure 2.5: *Hyperbolic tangent function*



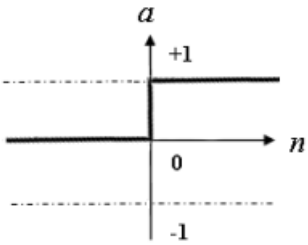
$a = \text{Tansig}(n) = \frac{2}{1+e^{-2n}} - 1$

Figure 2.6: *Purelin function*



$a = \text{Purelin}(n)$

Figure 2.7: *Hard-Limit function*



$$a = \begin{cases} 0 \text{ if } n < 0 \\ 1 \text{ if } n \geq 0 \end{cases}$$

C. Training Neural Networks

Training neural networks can be supervised or unsupervised. Supervised training is when inputs and their ideal output are provided to the network to be trained. Unsupervised training is when the neural network is not provided with the ideal output data.

In supervised training, the network is taught to produce the ideal output. As for unsupervised training, it usually teaches the neural network to group the input data into a number of groups defined by the number of neurons in the output layer.

Supervised and unsupervised trainings are both iterative processes. In the supervised training, each training iteration calculates the distance between the actual and the ideal output. This distance represents the error percent. Each iteration modifies the network's weight matrix to get the error to a low enough level. In the unsupervised training, due to the fact that there is no ideal output, we cannot measure the error, so we often iterate for a fixed number of iterations and then try to use the network. If the results are not satisfying, we provide the network with more training. There is a variety of neural networks training algorithm, each used for a specific network structure or task.

1. Back propagation algorithm

Back propagation algorithm is a popular form of supervised training mostly used by feedforward neural networks. This algorithm compares the ideal outputs against the outputs generated by the neural network. Using the compared values, an error is calculated. This error will be used by the back propagation algorithm to adjust the weights of the different layers in a backward manner, starting from the output layer until the input layer.

2. Simulated annealing

Simulated annealing is a faster way of training; it can work in conjunction with backpropagation to provide more accurate results, especially when backpropagation reaches local minima. A local minima is a false optimal weight matrix that prevents the backpropagation from continuing the training, considering that the best solution has been found.

The term annealing relates to the real life process of heating metals with high temperature to alter their properties, such as strength and hardness, and then cooling them slowly until they crystallize. Simulated annealing starts by randomizing the original values into greater degrees. During this process, the neural network is not allowed to increase the error rate, if so, then that cycle is discarded. When the original value degree starts decreasing, the weights fall into more optimal positions with a smaller error rate.

D. Neural Network Structure

Neural networks are made of groups of neurons, each group forming a layer. Most ANN has a minimum of two layers, the input layer and the output layer. The input layer receives the input data, and the output layer returns the output data or results. The input and output data are simply arrays of floating numbers. Between the input and the output layer, there are one or many hidden layers. It’s at the hidden layers level that the type of an ANN is dictated. Concerning the hidden layers, two decisions must be made at this level. The first is the number of hidden layers that should be present in the network, and the second is determining the number of neurons in each hidden layer.

The number of hidden layers depends mostly on the tasks expected from the neural networks. A neural network with no hidden layers is only capable of representing linear separable functions or decisions. One hidden layer can approximate arbitrarily any function containing a continuous mapping. Two hidden layers are rarely used; they can help conducting complex tasks with the neural networks, such as representing arbitrary decisions boundary to arbitrary accuracy with rational activation functions to approximate any smooth mapping to any accuracy.

Choosing the number of neurons in a hidden layer is a crucial part in constructing the neural network architecture. The neural network outputs are highly affected with the numbers of these neurons. A well fit neural network is one with an optimal number of neurons in its hidden layer. The number of neurons in a hidden layer can be approximated using some rule-of-thumbs methods such as:

- Choosing a number in the range between the input layer and the output layers sizes.
- The number should be equal to 2/3 of the input layer summed with the size of the output layer.
- The number should be less than twice the input layer size.

There are large numbers of ANN types. Each one has a specific structure and purpose, used to solve many kinds of tasks. For the sake of our study, we will concentrate on neural networks that can help us predict the market directions.

1. Feedforward neural network (Fig 2.8.a).

In the Feedforward neural network the input data is propagated in one direction directly towards the output layer with no kind of iteration or backward movements. The followings are two of many feedforward networks topology:

- Single-layer perceptron:

It is the simplest form of neural networks that consist of just one layer, the output layer. Weighted inputs are fed to the outputs layer. At each neuron, they are summed and compared to a threshold; if the value is bigger than the threshold, the neuron fires up and gives an output.

- Multilayer perceptrons or MLP :

Additional to the input and output layers, MLP have one or many hidden layers. In an MLP, neurons are processed with a nonlinear transfer function.

2. Recurrent neural network

Contrary to feedforward networks, recurrent networks contain iteration in their structure, and the input data is not propagated directly to the output layer. The topology of this network permits it to maintain a short-term memory, and to take into consideration past input effects on the present input.

- Elman neural network (Figure 2.8.b).

The Elman neural network is a simple recurrent neural network that consists of an input, a hidden and an output layer. The Elman neural network resembles a feedforward network, but it also has a context layer. This context layer is fed the output of the hidden layer without any weight. The context layer outputs these values on the next run of the neural network. The presence of this short-memory makes the Elman neural network very useful for predictions.

Figure 2.8.a: *Feedforward neural network*

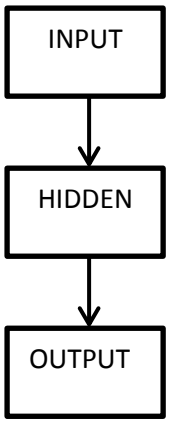
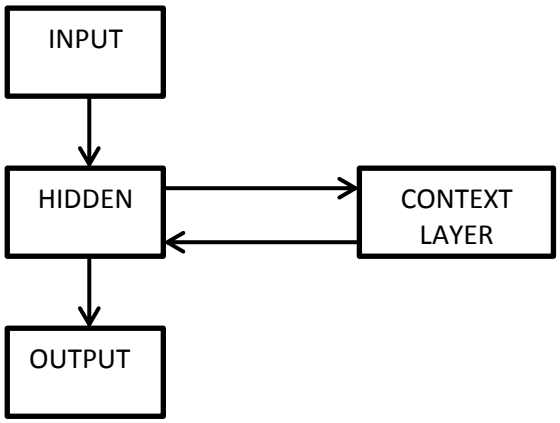


Figure 2.8.b: *Elman neural network*



E. Normalizing the Data

Usually, neural networks deal with a set of statistics; sometimes we try to use some of the statistics to predict others. Before constructing the network, a problem is defined in terms of an input array of floating points linked to an output array of floating points. The neural networks can't process the input and output arrays in their native form, they should be normalized before. Normalizing the data is the process of converting each input into a number between 0 and 1 or -1 and 1. To normalize the

inputs, we calculate the weight of each input by simply dividing it over a specific range.

F. A Financial Example

The following brief example will help us understand the above neural networks notions, and will give us a brief idea on how to feed the financial markets' data to neural networks in the purpose of predicting it. Let's consider predicting the foreign exchange market. Table 2.1 shows the EUR/USD closing prices over a specific range of time. The first step is normalizing the data. To do so, we compute the daily simple return using the following equation $r = \frac{Y_t - Y_{t-1}}{Y_{t-1}}$.

Table 2.1: *Normalized EUR/USD data*

Date	Time	EUR/USD	Normalized data
2012.09.24	0:00	1.29395	-
2012.09.25	0:00	1.28972	-0.00326906
2012.09.26	0:00	1.28767	-0.001589492
2012.09.27	0:00	1.29108	0.002648194
2012.09.28	0:00	1.2855	-0.004321963
2012.09.30	0:00	1.28057	-0.003835084
2012.10.01	0:00	1.28869	0.006340926
2012.10.02	0:00	1.291	0.001792518
2012.10.03	0:00	1.29216	0.000898528
2012.10.04	0:00	1.30114	0.006949604

After normalizing the data, we need to encode it in a certain format before presenting it to the neural network. The encoding depends on whether it's a recurrent or feedforward network. Since this example will describe both, let's start by the feedforward network format.

To predict the future price of the EUR, we must use "time windows". A time window contains the past prices that will be used to predict the future price. In this example, we are going to use a time window of four prices to predict the fifth, which means that the input layer should contain four neurons and the output layer one neuron. As for

the hidden layer, there are no rules for choosing their number of neurons; it's a matter of trial and error that sets them.

Now, we are able to create the supervised training's data by entering the simple return of four days, followed by an ideal output of what the next day is. Table 2.2 shows the training data:

Table 2.2: *EUR/USD training data*

input 1	input 2	input 3	input 4		output
-0.0033	-0.0016	0.0026	-0.0043	→	-0.0038
-0.0016	0.0026	-0.0043	-0.0038	→	0.0063
0.0026	-0.0043	-0.0038	0.0063	→	0.0018
-0.0043	-0.0038	0.0063	0.0018	→	0.0009

The encoding of the data differs for recurrent neural networks, and since there are several types of recurrent networks, we will be dealing with Elman networks. As described before, Elman networks contain a short term memory that enables them to memorize past values and use them in future training iterations. We will be using an Elman network with a single input neuron and a single output neuron, which means that we will provide a single day return to forecast the next day's price. The network will also have a hidden layer such as the one present in the feedforward network. This hidden layer will be linked to a context layer so that they can exchange data. Similar to the hidden layer, the context layer neurons' number is a matter of trial and error. Table 2.3 shows the Elman network training data.

Table 2.3: *Elman network training data*

Input		output
-0.0033	→	-0.0016
-0.0016	→	0.0026
0.0026	→	-0.0043
-0.0043	→	-0.0038
-0.0038	→	0.0063
0.0063	→	0.0018
0.0018	→	0.0009
0.0009	→	0.0069

In an Elman network, the order is very important. Tomorrow's price cannot be predicted using today's price alone. Rather, due to the presence of the context layer, the last days will influence the output.

The Average Directional Index (ADX)

A. Introduction

Developed by Welles Wilder, the ADX is a financial indicator that measures the strength of a trend independent of its direction. The ADX is complemented with two other indicators, Minus Directional Indicator (-DI) and Plus Directional Indicator (+DI) that defines the trade direction. Used together, these indicators can help investors and traders measure the trend's strength and direction.

A positive directional movement (+DI) is created when the variation of two highs over a period of time T is greater than the negative variation of the lows of this same period. Negative values are considered null.

A negative directional movement (-DI) is created when the variation of two highs over a period of time T is smaller than the negative variation of the lows of this same period. Negative values are considered null.

The +DI and –DI calculations can be expressed with the following equations:

If Δ High(T) > - Δ Low(T) then +DI= Δ High(T)

If Δ High(T) < - Δ Low(T) then -DI= - Δ Low(T)

If Δ High(T) < 0 then +DI= 0

If - Δ Low(T) < 0 then -DI= 0

The calculation of the ADX is beyond the scope of this study, we will focus more on its interpretation.

B. Interpretation

The basic use of the ADX is detecting if a market is trending or not. Such an indicator would be very informative for trend followers and trend following systems.

After specifying its optimal parameters (usually 14 or 20 days), the ADX outputs the resulting value. When its value reaches 25 or above, the ADX indicates the presence of a strong trend, and when its value is below 20, it indicates that the market is sideways.

C. DI Crossover Strategy

When the ADX value is above 25 (this value depends on the traders' expectations) indicating that the market is trendy, a long signal is generated when the +DI crosses the –DI, and a stop loss is set on the low of the signal day. This bullish signal is reinforced when the ADX's value increases. In the same manner, a short signal is generated when the –DI crosses above the +DI, and a stop loss is set on the high of the signal day. This bearish signal is also reinforced when the ADX increases.

Stochastic oscillator

The stochastic oscillator is a momentum indicator that compares a security's closing price relatively to its price range over a time period.

A. Calculation

To calculate the stochastic oscillator we use the following formulas

$$\%K = \frac{C - L}{H - L} * 100$$

C is the current closing price,

L is the lowest low of the previous N period.

H is the highest high of the previous N period.

%D= 3-day SMA of %K

William %R

Considered as the inverse of the stochastic oscillator, Williams %R is a momentum indicator that measures the level of the closing price relative to the highest high.

A. Calculation

$$\%R = \frac{H - C}{H - L} * -100$$

C is the current closing price,

L is the lowest low of the previous N period.

H is the highest high of the previous N period.

Value at Risk (VaR)

VaR is mainly a market risk measurement technique. It is considered as an estimate of the amount that a trader or investor could lose due to market fluctuations during a specific holding period. This measure can be used to assess the risk, set margin requirements and choose the trading positions' size. In either case, VaR can prevent huge losses due to an extreme event. From an investor or trader view, VaR can be defined as the maximal loss of a financial position during a specific period for a specific probability.

A. VaR Calculation

There are two approaches for calculating VaR. The following is a description of both.

1. Historical Simulation

Historical simulation is a popular way of calculating VaR. Past data are used as a guide to estimate what may happen in the future. Suppose that we wish to calculate the VaR or expected VaR of a EUR trade using a one day time horizon, a 99% confidence level and 200 days of past data. From these 200 days' prices, we can simulate 200 different scenarios about how tomorrow's price might be. Scenarios are created by measuring the percentage changes of the sample's consecutive values, and then each scenario is created by multiplying a certain measured percentage change

with the price of the day on which we collected the data. After creating the scenarios, we calculate the dollar change between today and tomorrow for each scenario, thus a probability distribution is created. The VaR estimation is none other than the one percentile point of this distribution. Table 2.4 shows VaR calculated using the historical simulation. Figure 2.9 shows the historical model's sample distribution.

2. Model Building Approach

Model building approach is an alternative to historical simulation. Let's take the same example used in the historical simulation to highlight the differences between both approaches. First of all, the 200 price sample should contain the most recent EUR prices. We compute the volatility of daily changes. In case we're dealing with a portfolio of assets, then a variance/covariance matrix is constructed to compute the volatility. After computing the volatility, the average value of the dollar's daily changes is calculated. Assuming that the dollar price changes are normally distributed, the 99% estimation of the VaR is none other than the difference between the computed average and the volatility multiplied by 2.33 (2.33 is the number of sigma associated with a probability of 1% in a normal distribution table). Table 2.5 shows VaR calculated using the model building approach. Figure 2.10 shows the model building approach's sample distribution.

In both the above approaches, the VaR time horizon is one day. To estimate data over periods of time longer than one day, the following equation is used:

N-day VaR = 1-day VaR * $\sqrt{N}$

N is the number of days to estimate VaR for.

Table 2.4: *VaR using the historical simulation*

Long 100,000 EUR, VaR(1%) = 2,857 USD

Day	Date	Eur	scenario	Date	Eur	Deur	Dptf
0	12/5/2008	1.2734	1	Tomorrow	1.479808	0.023107696	2,310.77
1	12/8/2008	1.2936	2	Tomorrow	1.453997	-0.002702597	(270.26)
2	12/9/2008	1.2912	3	Tomorrow	1.46832	0.011620206	1,162.02
3	12/10/2008	1.3015	4	Tomorrow	1.494978	0.038278248	3,827.82
4	12/11/2008	1.3357	5	Tomorrow	1.458336	0.001635884	163.59
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
200	9/11/2009	1.4567	201	Tomorrow	-	-	-

Figure 2.9: *Historical Model Sample Distribution*

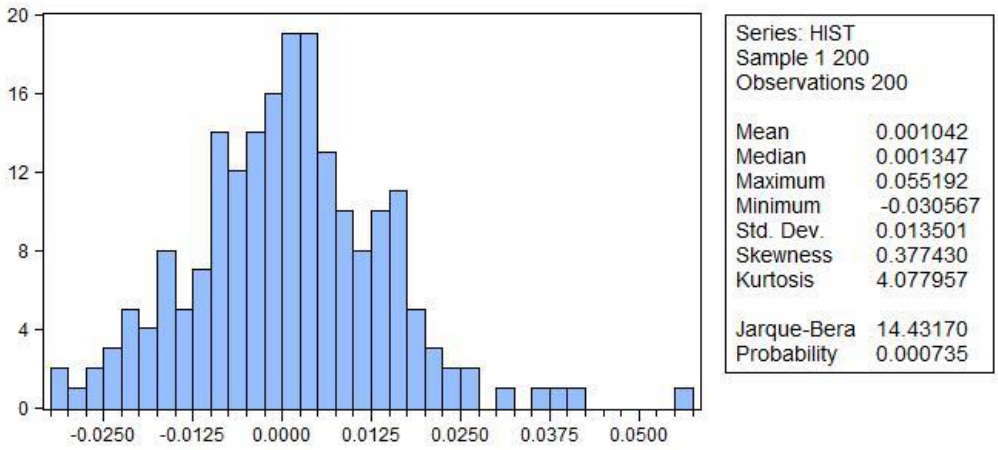
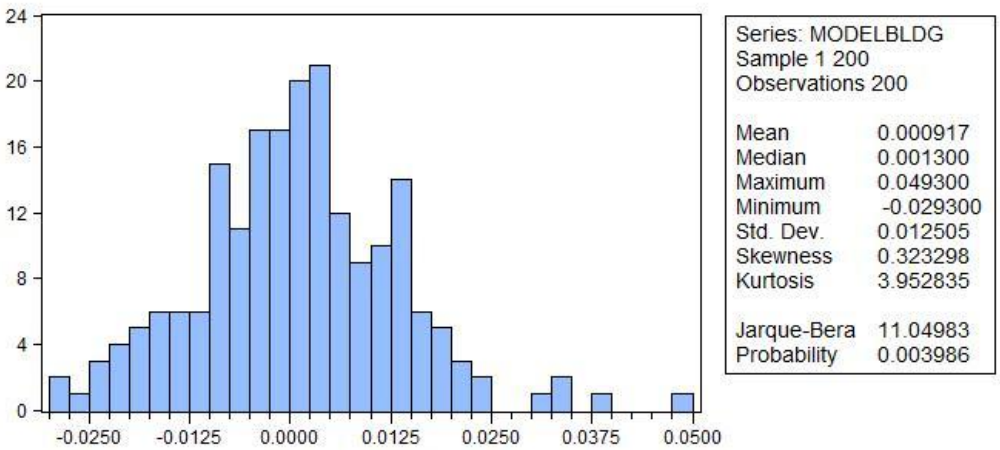


Table 2.5: *VaR using the model building approach*

Long 100,000 EUR, VaR(1%) = 2,822 USD

Day	Date	Eur	Deur	Dptf		
0	12/5/2008	1.2734	NA	NA	Mean	91.65
1	12/8/2008	1.2936	0.0202	2020	stdev	1250.54994
2	12/9/2008	1.2912	-0.0024	-240		
3	12/10/2008	1.3015	0.0103	1030		
4	12/11/2008	1.3357	0.0342	3420		
⋮	⋮	⋮	⋮	⋮		
200	9/11/2009	1.4567	-0.001	-100		

Figure 2.10: *Model Building Approach Sample Distribution*



Chapter 3

The Implementation

After introducing our trading plan, as well as all the methods and techniques used to build our trading algorithm in chapter 2, we will start developing the algorithmic trading system. As discussed earlier the trading system development consists of four stages, the first two of which were processed in the previous chapter. This chapter will concentrate on the implementation stage.

The Electronic Exchange Platform

As mentioned earlier, this study will not deal with developing an electronic exchange platform, but will rather be concentrating on developing a trading algorithm using an already available electronic exchange platform.

One of the most advanced and widely used electronic exchange platforms that allow algorithm development and algorithmic trading is the Metatrader 5. A full Integrated Development Environment (IDE) is included in the Metatrader 5. The IDE includes all the necessary programming tools, methods and functions that allow creating, debugging, testing, optimizing and running an algorithmic trading system. Figure 3.1 shows the Metatrader 5 platform.

The core of the IDE is the MetaQuotes Language 5 (MQL5), which is a high level programming language based on the popular C++ programming language that permits the creation of trading algorithms (known as Expert Advisors), custom financial indicators, scripts and libraries.

Expert Advisor: it is a trading algorithm that automates trading on the market. Expert Advisors are very powerful; they can access all the analytical information and trading functions, they also analyze the market and execute trading orders.

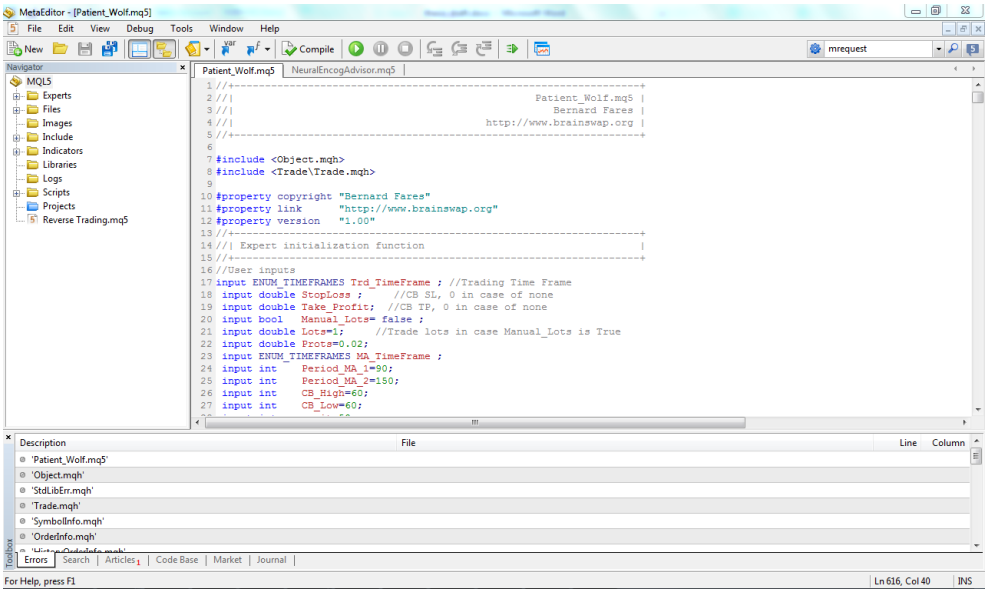
Custom Indicator: it is a technical indicator that can be created by developers or traders to suit their analytical needs. Custom Indicators cannot access trading functions. Instead, they are used in Expert Advisors to help take the trading decisions.

Script: it is a developed program that performs a single task. This program is able to access trading functions only once, unlike Expert Advisors that can execute them on each new tick.
Library: it is a set of custom functions which can be used commonly between Expert Advisors, Custom Indicators and Scripts.

MQL5 also contains built-in indicators and functions that help control and open trading positions.
The MQL5 code is written in a special editor shown in figure 3.2, called MetaEditor, and in which the code is compiled and checked for errors.
One of the most important features of Metatrader 5 is the strategy tester. It allows us to test each developed strategy and back-test it on any chosen market with any range of data. Furthermore, the strategy tester contains an optimization module that can test the same strategy on many markets and compute the most suitable one. It can also compute the best parameters that suit our objectives, which means that if we are seeking a minimum drawdown for example, the strategy tester will compute the best parameters for it. The strategy tester is available in two modes: a visual mode that shows the tester a simulation of his strategy with the market as if it was ran in real-time and all the used indicators placed on the chart, and a none visual mode used to just compute the results without a visual simulation. The Metatrader 5 results contain performance measures, P&L statistics and many others we will later discuss.

Figure 3.1: *Metatrader 5*

Figure 3.2: *MetaEditor*



The Strategy

Algorithm uses two different strategies based on the market conditions. These conditions are detected using the ADX indicator and the ADX's critical value set by the user.

If the ADX Value > A, then a trend is growing strong and a channel breakout strategy is used. Under these circumstances, the following conditions are applied:

- If the previous closing price P_{t-1} is higher than the highest high of the most recent N period of time, then a buy signal and a sell closing signal are generated.
- If the previous closing price P_{t-1} is lower than the lowest low of the most recent N period of time, then a sell signal and a buy closing signal are generated.
- If the previous closing price P_{t-1} is higher than the highest high of the most recent K period of time (where K<N), then a sell closing signal is generated. This signal is known as exit signal.
- If the previous closing price P_{t-1} is lower than the lowest low of the most recent K period of time (where K<N), then a buy closing signal is generated. This signal is known as exit signal.

If the ADX Value < A, then the market is tending to be sideways, and a strategy based on an artificial neural network's predictions will be used. Under such circumstances, the following conditions are applied:

- The ANN will be forecasting the following day return. If the forecasted return $r_t > 0$ and $r_t > x$ (where x is the minimum required return), then a buy signal and a sell closing signal are generated, and a take profit is set on $P_t*(1+r_t)*90\%$.
- If the forecasted return $r_t < 0$ and $r_t < x$ (where x is the minimum required return), then a sell signal and a buy closing signal are generated, and a take profit is set on $P_t*(1+r_t)*90\%$.

Modeling the Algorithm

The model in figure 3.3 shows the process conducted on each new tick.

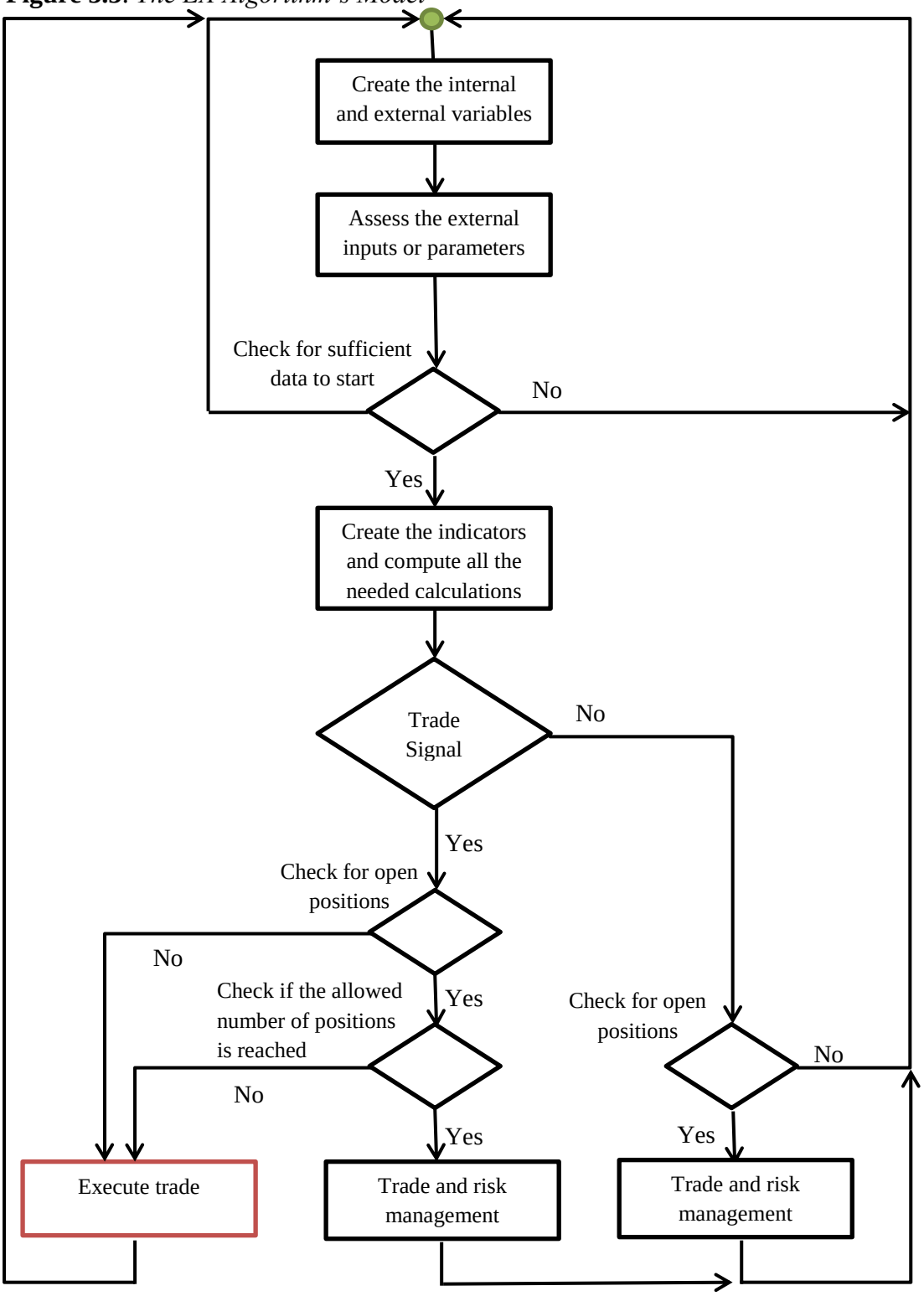
At first, the Expert Advisor (EA) starts by creating and allocating the memory for each variable that will be used, then the external parameters and settings set previously by the user are fed to the EA. All the introduced external variables are

evaluated and tested for consistency. Before creating the indicators using their specified parameters, the EA makes sure that all the needed data is available. If the data isn't enough, the EA loops back to the start, and it keeps on looping back with each tick, till the required data is met. If the required conditions are met, the EA creates the indicators and starts by applying the previously described strategy to the market. At this level the EA outputs a trade signal if there is any.

If a trading signal is generated at this level, then the EA checks for already opened positions; in case there aren't any, an order is executed, and a short or long position is opened depending on the generated signal. But, if there already are open positions, the EA checks if the number of allowed positions or positions' sizes (defined in the strategy) has been reached. If so, then no new orders are executed, and the EA performs a trade and risk management to assess the risk and modify opened positions. Otherwise, an order is executed.

If the EA doesn't provide any trading signal, an open position check is carried out. In case there were any opened positions, a trade and risk management is conducted.

Figure 3.3: *The EA Algorithm's Model*



The Coding

The Coding starts by assessing all the external variables that will be used in the development. A special user friendly tab shown in figure 3.4 is created for that purpose. A brief description of each input provided by the user can help understand the algorithm and its flexibility.

Trd_TimeFrame: this variable is used to indicate the trading time frame; all time frames are available and selected from the TimeFrame combo box.

StopLoss: this variable is used to indicate the stop loss of each trade. The stop loss value inserted in this text box is the number of pips away from the original trading price. The value 0 indicates that no stop loss is set.

Take_Profit: similar to the stop loss, this variable is used to indicate the take profit of each trade. The take profit value inserted in this text box is the number of pips away from the original trading price. The value 0 indicates that no take profit is set.

Manual_Lots: if set to false, the algorithm will automatically choose the size of each trade based on the market's volatility. If set to true, the user should manually place the size of the trades.

Lots: it is the size of the trades placed manually by the user.

CB_High: it is the channel breakout period parameter for which a long position is signaled.

CB_Low: it is the channel breakout period parameter for which a short position is signaled.

Exit: it is the channel breakout period parameter for which a position is closed.

ADX_TimeFrame: this variable is used to indicate the ADX time frame; all time frames are available and selected from the TimeFrame combo box.

Adxperiod: it is the period used to compute the ADX indicator.

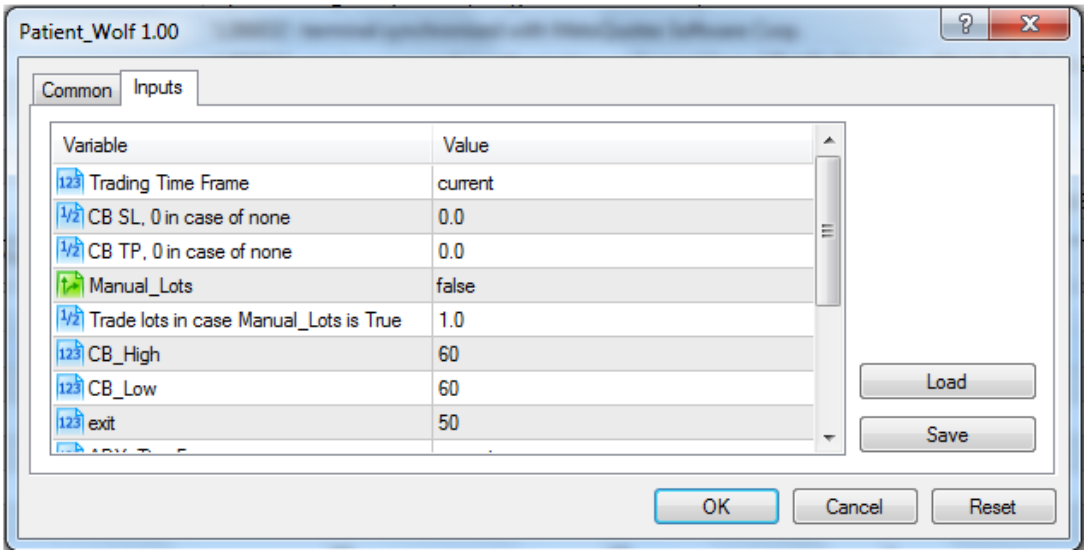
ADX_SValue: it is the ADX value for which the market is considered forming a trend.

EA_Magic: each Expert Advisor is identified by a number called magic number. Each trade initiated by the EA is marked with this number which is set by the user.

VaR_TimeFrame: it is the time frame for which the VaR is calculated; all time frames are available and selected from the TimeFrame combo box.

VaR_Period: it is the period for which the VaR is calculated.

Figure 3.4: *User Inputs' Window*



After creating and allocating the external variables, the internal variables and the handle of each indicator are also created. Then, based on the external variables, the market is tested to see if there is enough data to build all the indicators. This test is conducted by counting the bars of each time frame specified in the external variables and comparing them to the periods specified for each indicator.

If all the needed data is present, we start creating the indicators.

- The ADX indicator: this indicator is already built in the Metatrader 5, so we should initialize it and copy its values to a dynamic array.
- The channel breakout: the market's highs and lows of an N period (set by the user) are computed. These two computed values are used to provide the trading signal.

  The market's highs and lows of a k period (set by the user) are also computed and used to provide the exit signal.
- The ANN indicator: a custom indicator is created and developed using the ENCOG framework. Based on three inputs, Stochastic % K, Stochastic % D and Williams % R, the following day's expected return is plotted on the chart and used to provide trading signals.
- VaR: the VaR time frame and period x are predefined using the external variables specified by the user. So the x closing prices of the specified time frame are

copied to an array, then the return of each closing price is computed using the formula $r = \frac{P_t - P_{t-1}}{P_{t-1}}$, where P is the closing price and t the time period.

After computing the return, we multiply it by 100000 (the size of 1 lot), then we compute the standard deviation and the mean for the resulting series. Then, the 99% VaR is none other than the difference between the mean and standard deviation multiplied by 2.33. The VaR is shown on the chart as a comment.

After that, all the indicators are created and initialized, and generating values that are buffered into arrays. The market should be then checked for any trade opportunity; if any is found, four Boolean variables interfere.

- Opn_B: when its value is set to true, a signal to open long positions is generated. Otherwise, no signal is generated.
- Cls_S: when its value is set to true, a signal to close all short positions (if there are any) is generated.
- Opn_S: when its value is set to true, a signal to open short positions is generated. Otherwise, no signal is generated.
- Cls_B: when its value is set to true, a signal to close all long positions (if there is any) is generated.

If signals are generated, positions must be opened or closed. To do so, the orders should be sent either directly to the exchanges or to the brokers so that they can be routed to the exchanges. This process is handled by Metatrader 5 using a set of built-in functions. Orders are sent respecting the markets' protocol and standards. The MT5 platform supports all types of orders (instant execution and all sort of pending orders). Each sent order should be equipped with its type (short or long), the order type (instant or pending), the EA magic number, the trading symbol, the trading price, the trade size, the stop loss, the take profit and the allowed deviation from the specified price.

Before requesting any order, the EA should check for already opened positions. This is done by counting the total positions in the Metatrader 5's terminal. In case there were none, the EA proceeds with the order request and sends the desired order to its meant destination. But if there were other positions detected in the terminal, these positions' details should be assessed. By position details we mean the position type (short or long), the order type (instant or pending), the trading symbol, the trading price, the trade size, the stop loss, the take profit and the P&L of each position. These

details will be used later for the trading and risk management. Before proceeding to the trade request, the EA checks if the number of allowed orders has been reached. If so, opening a new position is forbidden and the EA should return to the start. If the allowed number of orders hasn't been reached, the trade request should be executed. For the sake of our strategy, just one order is allowed in the terminal.

Implementing the ANN

The Artificial Neural Network is implemented into the EA using ENCOG from Heaton Research. Available in Java, C/C++ and .Net, ENCOG is an advanced ANN framework. It contains classes and methods used to create a wide variety of networks. ENCOG also supports normalizing and processing the data.

A. Why Choose ENCOG?

- ENCOG has already been used in other trading software packages. This means that it has already been tested to predict financial time series.
- One of ENCOGs best advantages is that it is an open source software. All the source code used to create the ANN and process them are available and subject to changes. The source code is available in C# and Java.
- Heaton Research did a great job in documenting ENCOG and how to use it. They even went further and created a wiki that explains the different types of networks and how to use them with ENCOG.
- ENCOG is always under continuous development.
- ENCOG supports multiple processors and multithreading, which leads to quicker neural network calculations.

B. The ANN Process

After choosing ENCOG which is the framework that we will be using to process our ANN calculations, six other problems should be handled.

1. What data should we use and how should we introduce it to the ANN ?
2. What ANN architecture should we use?
3. What type of training should we use?
4. How should we use the generated outputs ?

The following will deal with all the mentioned tasks.

C. Creating the Inputs

In order to start creating our ANN, we should define the inputs and the expected outputs. Since technical indicators are used by traders in conjunction with the market's prices to predict the future sentiment or direction of the market in order to place trades, then we will try to achieve the same with ANN. The ANN will be fed indicators output, and expected to give us the market's forecasted return. The used indicators are Stochastic %K, Stochastic %D and William %R. In Metatrader 5, a script that handles creating the stochastic indicators then time-box their values is developed. Time-boxing is the process of saving the indicators' generated values in a timely basis manner, so they could be introduced to the network as moving slices of data. As mentioned earlier, the data needs to be normalized. Fortunately, this process is handled by ENCOG, so we do not need to implement such module in our developed script. Finally, the developed script will export all the gathered data to a CSV file so it could be introduced to ENCOG where it will be processed.

D. The Neural Network Structure

A feedforward network will be used to conduct such predictions. The network will be made up of three layers: an input layer, a hidden layer and an output layer.

The input layer's number of neurons will be equal to the sum of each indicator times the number of bars used to predict the next bar.

The hidden layer's number of neurons will be estimated using the rule of thumb mentioned earlier, and they are subject to change to minimize the error rate. All the neurons in the hidden layer will be using a hyperbolic tangent's function.

The output layer will be equal to the number of bars we are predicting, and since we are predicting the closing price of just one bar, then the output layer will contain only one neuron that uses a Purelin function.

E. Training The ANN

A backpropagation algorithm is used in ENCOG to train the CSV file imported from Metatrader5. ENCOG has a special training console where all the training parameters are available. The network could be trained for a certain period of time or until the error rate between the generated and expected outputs reaches a certain value. In our case, we will be going to train the network for 3 hours.

F. Using the Trained Neural Network

The resulting outputs are generated and exported to a CSV file. The generated CSV file will be used in MT5 to create a custom indicator that uses a developed DLL to communicate with the ENCOG functions and predict the return of one bar in the future. Creating the indicator consists of two parts. The first is sending the indicators' values to ENCOG so they could be used as inputs for the network. The second part involves migrating the outputted results using a DLL from ENCOG to MQL5 so they could be used to build the indicator.

Chapter 4

The Empirical Study

While developing and implementing the algorithmic trading system, each indicator or method used has one or several input parameters that are declared as external variables and are set by the trader or investor. Figure 3.4 shows our system's parameters which could be indicators' time periods, price ranges, weights, etc. The best parameters are the ones that generate the best results and highest performance with the minimal drawdown. In this chapter, we are going to find out the best generalized parameters that suit our system and how to compute them. Finally, the system will be back-tested and the results interpreted.

Generalization vs. Optimization

Generalization and optimization are often mistaken for one another, yet they are two different concepts used to choose the algorithmic trading system's parameters, and each one has a different impact on the trading results.

Generalizing a system's parameters is the process of choosing parameters that suit the future long term pace of the market and can result in future profits on long future time periods. Finding such parameters consists of conducting back-tests on long time periods (more than one year) and choosing the parameters that proves to be profitable and performant on the entire back-tested period. Generalized parameters do not have to be the ones that generated the best profits on the back-test results. Instead, they should be the ones expected to generate a steady profit on the long term future time period.

Optimizing a system's parameters is the process of choosing parameters that will probably suit the near short term market, and that will probably result in future short-term profits. Finding such parameters consists of conducting back-tests on the most recent short time period and choosing the parameters that generate the best profits.

Our system's objective is to run for long periods of time, generating profits with minimal drawdowns and higher performance. Thus, all our parameters need to be generalized and expected to continuously generate profits in the near and long future.

The Drawdown

Among all the back-testing results, the maximal drawdown is considered a critical measure. The drawdown is the peak-to-trough decline of the balance or the equity during the back-tested period. Drawdowns help in determining our systems' financial risk. While back-testing the system for the best generalized parameters, we aim to maximize our profits and minimize the maximal drawdown.

Metatrader 5's Strategy Tester

To test how our algorithm will be conducting trading operations, we will be using the Metatrader 5's built-in strategy tester. This advanced tool uses the market's historical data, then goes through the quotes and analyses them one by one. During this process, all the orders are virtually executed, which allows simulating the algorithm behavior during the past. The algorithm can be tested on multiple financial instruments and later evaluated.

The strategy tester offers several testing modes. The algorithm can be tested using every tick in the historical data to ensure the best accuracy, or using simulated conditions, such as using a 1 minute OHLC or open prices only to provide faster tests.

The tests results are not limited to displaying the algorithm profits, they also display a great amount of statistical data including P&L, maximum drawdowns, performance ratios, risk factors and much more.

In order to help traders, investors and developers to be more accurate, the strategy tester permits live monitoring while back-testing the algorithm, which means that we are able to track the algorithm operation on historical price data as if it was ran in real time.

As mentioned earlier, the strategy tester permits optimization. During this process, the algorithm is tested multiple times with different sets of parameters, then the results are compared and the best performant parameters are selected.

Training the Network

Using ENCOG, each pair of currencies used later in the back-testing has its own network. All the built neural networks has the following settings showed in figure 4.1.

Figure 4.1: *The ANN settings*

```
/// <summary>
/// The size of the input window.  This is the number of bars used to predict the next bar.
/// </summary>
public const int INPUT_WINDOW = 15;

/// <summary>
/// The number of bars forward we are trying to predict.
/// </summary>
public const int PREDICT_WINDOW = 1;

/// <summary>
/// The number of bars backward to look for the best result.
/// </summary>
public const int RESULT_WINDOW = 5;

/// <summary>
/// The number of neurons in the first hidden layer.
/// </summary>
public const int HIDDEN1_NEURONS = 7;
```

As mentioned earlier, the ANN of each pair will be trained for 3 hours. The error rate of the trained networks ranged between 4.4% and 6.9%. The process of training is shown in figure 4.2.

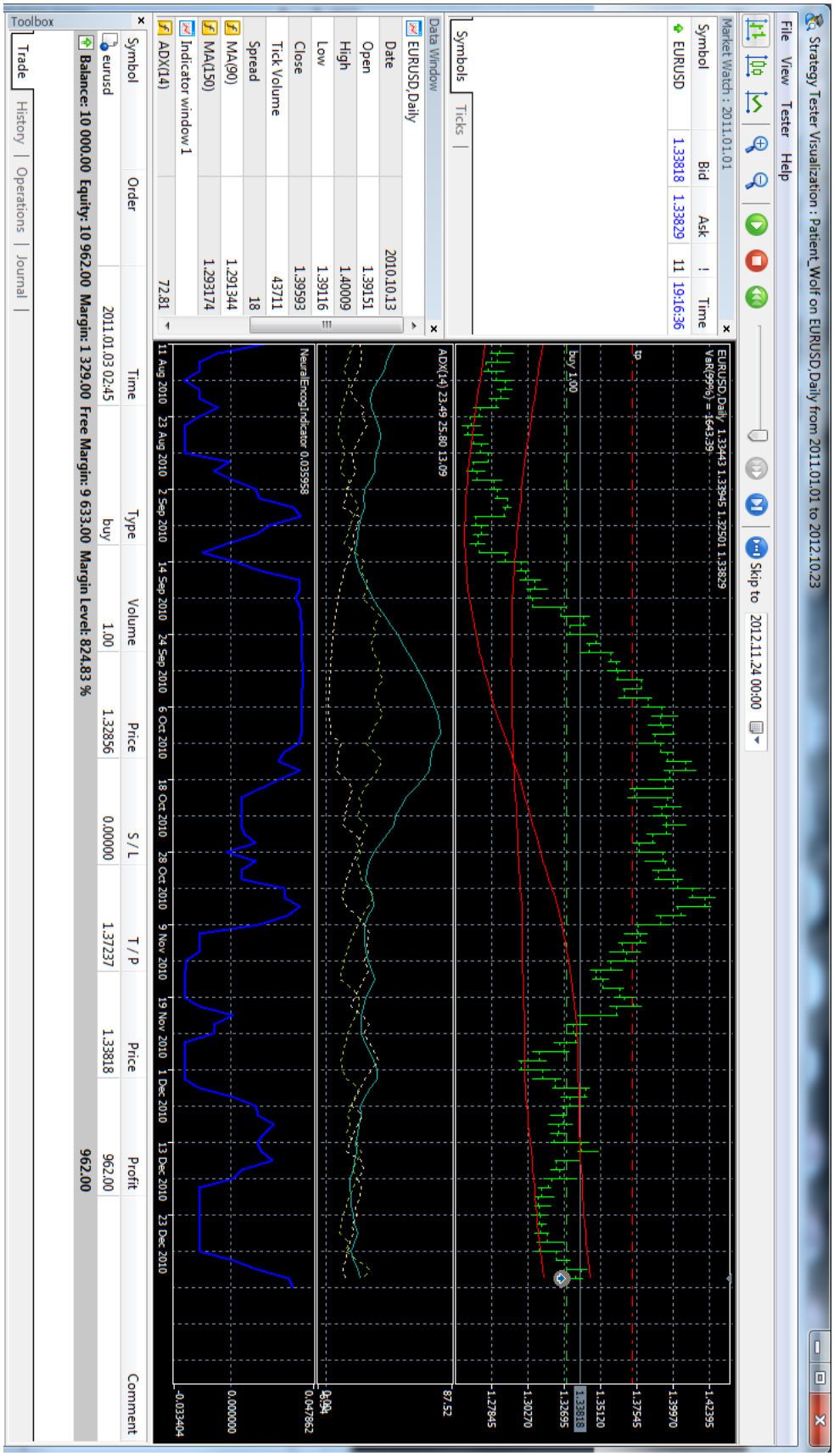
Figure 4.2: *Training the ANN*

```
C:\MetaTrader5\EncogCSharp\EncogCSharp\Indicators\Indicators\bin\Debug\Indicators.exe
Iteration #970,258 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,259 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,260 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,261 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,262 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,263 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,264 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,265 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,266 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,267 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,268 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,269 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,270 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,271 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,272 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,273 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,274 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,275 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,276 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,277 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,278 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,279 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,280 Error:5.877627% elapsed time = 01:30:36 time left = 01:30:00
Iteration #970,281 Error:5.877406% elapsed time = 01:30:36 time left = 01:30:00
```

The Tests

Before starting the sets of back-testing, we should mention that all the tests are conducted with an initial amount of 10,000 USD with a leverage of 1:100, and each trade's size will be 1 Lot. A daily time frame will be used with custom periods that vary with each test. Figure 4.3 shows how the visual back-test is conducted and it also shows the different implemented indicators.

Figure 4.3: *The Visual Back-test*



A. Preliminary Tests

The preliminary tests consist of choosing the best pair of currencies that suits our algorithm and can probably generate the best profits with the minimal risk. For conducting such tests, we will be using a set of indicators' parameters that we have found by research, and that are proven to be profitable when the indicator has been used by other investors. Figure 4.4 shows the used parameters.

Figure 4.4: *The External Parameters*

Variable	Value
CB SL, 0 in case of none	0
CB TP, 0 in case of none	0.0
Manual_Lots	false
Trade lots in case Manual_Lots is True	0.5
CB_High	60
CB_Low	60
exit	50
ADX_TimeFrame	1 Day
adxperiod	14
ADX critical Value that indicates if the market is trending	25
Expert Magic number	0
VaR_TimeFrame	1 Day
VaR_Period	100

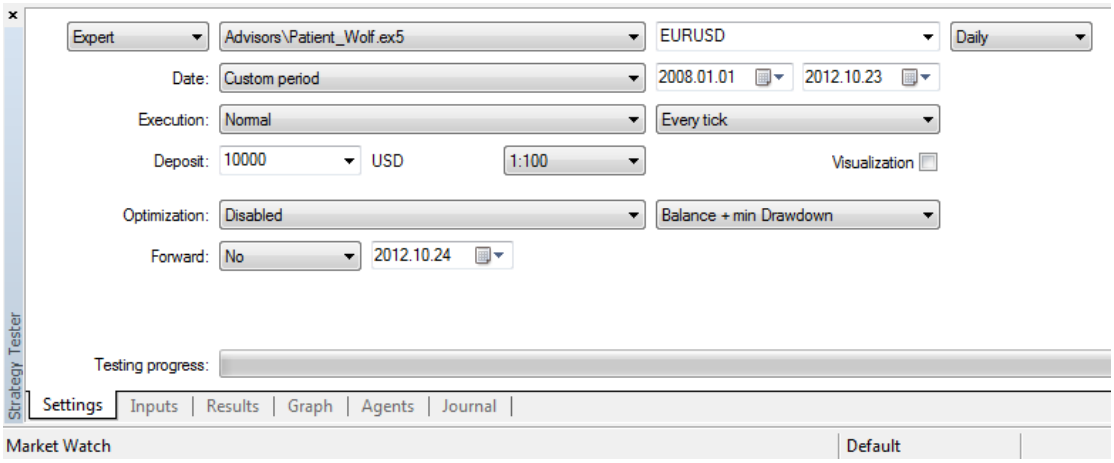
Strategy Tester

Settings | Inputs | Optimization Results | Optimization Graph | Agents | Journal

The test is going to be performed on three pairs: EURUSD, AUDCAD and USDCAD. A neural network is formed for each pair of currencies. As for the time period, the test is conducted from 01/01/2008 till 10/23/2012.

Figure 4.5 shows the strategy tester's layout and how to set the different settings.

Figure 4.5: *The Strategy Tester*



After conducting the back-tests on the mentioned currency pairs, the results are shown in table 4.1.

Table 4.1: *Preliminary Test Results*

Symbol	Profit	Expected Payoff	Profit Factor	Recovery Factor	Sharpe Ratio	Equity DD %
AUDCAD	5698.85	20.95	1.13	0.75	0.06	45.62
EURUSD	19,950.5	69.51	1.32	1.62	0.12	40.25
USDCAD	6432.46	21.76	1.18	0.77	0.07	42.93

As we see, a preliminary test on the EURUSD over a period of approximately 4 years resulted in great profits and acceptable maximal drawdowns. As for the other currencies compared to the EURUSD, they generated less profit with approximately the same level of drawdown or risk. Thus, back-testing the system on the EURUSD pairs outperformed other currencies, and for this particular reason we are going to use this pair for all our upcoming back-tests.

For the sake of our study, the detailed results of the EURUSD preliminary results are shown in figures 4.6.a, 4.6.b and 4.6.c.

Figure 4.6.a: *EURUSD Preliminary Test's Balance and Equity Curve*

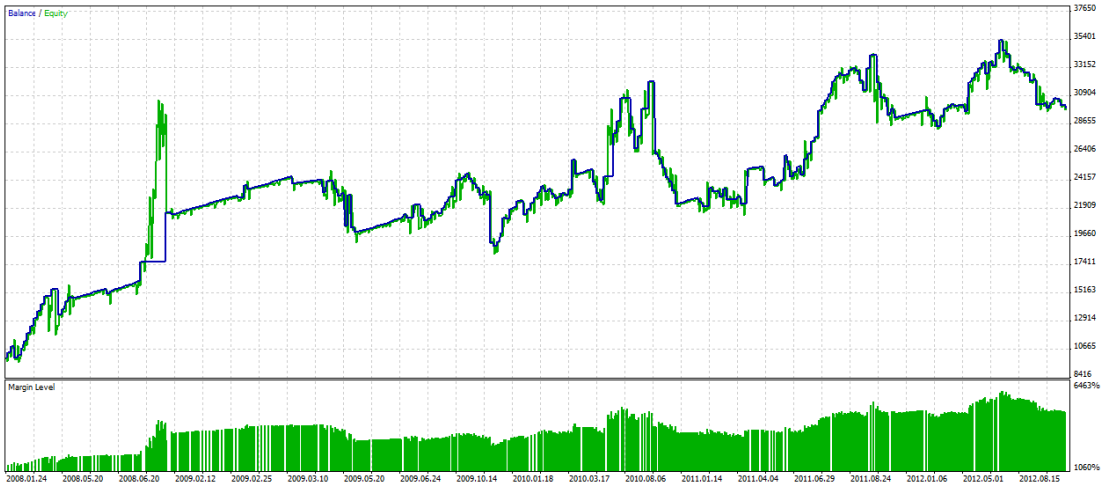


Figure 4.6.b: *Preliminary EURUSD Test Results*

History Quality	100%				
Bars	1250	Ticks	70109434	Symbols	1
Initial Deposit	10 000.00				
Total Net Profit	19 950.50	Balance Drawdown Absolute	0.00	Equity Drawdown Absolute	297.35
Gross Profit	83 209.05	Balance Drawdown Maximal	9 957.00 (31.08%)	Equity Drawdown Maximal	12 342.65 (40.25%)
Gross Loss	-63 258.55	Balance Drawdown Relative	31.08% (9 957.00)	Equity Drawdown Relative	40.25% (12 342.65)
Profit Factor	1.32	Expected Payoff	69.51	Margin Level	1325.50%
Recovery Factor	1.62	Sharpe Ratio	0.12	Z-Score	-2.86 (99.58%)
AHPR	1.0045 (0.45%)	LR Correlation	0.87	OnTester result	0
GHPR	1.0038 (0.38%)	LR Standard Error	2 751.44		
Total Trades	287	Short Trades (won %)	131 (65.65%)	Long Trades (won %)	156 (74.36%)
Total Deals	574	Profit Trades (% of total)	202 (70.38%)	Loss Trades (% of total)	85 (29.62%)
	Largest	profit trade	3 916.00	loss trade	-5 758.05
	Average	profit trade	411.93	loss trade	-744.22
	Maximum	consecutive wins ($)	19 (1 472.50)	consecutive losses ($)	7 (-9 720.95)
	Maximal	consecutive profit (count)	8 164.40 (4)	consecutive loss (count)	-9 720.95 (7)
	Average	consecutive wins	4	consecutive losses	2

The following figures are defined by MetaQuotes and available in the testing report:

History Quality: this value characterizes the quality of price data used for testing. It is determined as a percentage ratio of correct and incorrect minute data. Incorrect minute bars are the ones having zero spread, or volume 1 when OHLC values are different. History gaps are also considered as incorrect data. Depending on size, the period of testing is divided into 1 — 199 intervals. Then the history quality is determined for each of them separately. The time intervals are shown on the graphical indicator of the history quality (the lighter tint of green means the better quality, the red color represents intervals with the quality lower than 50%).

Bars: number of bars generated for the symbol the testing is conducted on.

Ticks: number of ticks during testing.

Symbols: the number of symbols, for which information was requested by the Expert Advisor during testing.

Initial Deposit: the initial deposit for testing.

Withdrawal: a sum of money withdrawn by an Expert Advisor during testing. This field is not displayed if there are no withdrawal operations.

Total Net profit: the financial result of all trades. This values is the difference between the Gross Profit and Gross Loss.

Gross Profit: sum of all profitable trades in monetary units.

Gross Loss: sum of all losing trades in monetary units.

Balance Drawdown Absolute: the maximal fall of balance below the initial deposit.

Balance Drawdown Maximal: the maximal fall of balance relatively to the local maximum in the deposit currency and as percent from the deposit.

Balance Drawdown Relative: the maximal fall of balance relatively to the local maximum of balance in percent, and the appropriate money value.

Equity Drawdown Absolute: the maximal fall of equity below the initial deposit.

Equity Drawdown Maximal: the maximal fall of equity relatively to the local maximum in the deposit currency and as percent from the deposit.

Equity Drawdown Relative: the maximal fall of equity relatively to the local maximum of equity in percent, and the appropriate money value.

Profit Factor: ratio of the gross profit to the gross loss. One means that the sum of profits is equal to the sum of losses.

Recovery Factor: this value reflects the riskiness of the strategy - the amount of money risked by the Expert Advisor to make the profit it obtained. It is calculated as the ratio of a profit obtained to the maximal drawdown.

AHPR: arithmetic mean of a trade (change in percent). Arithmetic mean of equity changes per trade. The arithmetic mean usually overestimates the profitability of a trading system as compared to the geometric mean. If the geometric mean implies the multiplication of each trade results, the arithmetic mean just sums them. The value in percent is given in brackets. It is positive if the trading system is profitable. The negative value means that the system is losing.

GHPR: geometric mean of a trade (change in percent). Geometric mean shows by how many times did the capital change after each trade in average. The relative equity change is often a more objective estimation than the expected payoff. Capital change in percent is given in brackets. Negative number in brackets means that the capital decrease in average on each trade.

Expected Payoff: this is a statistically calculated value showing the average profitability/loss of one deal. This value can also be considered as a forecasted profitability/loss of the next deal.

Sharpe Ratio: this ratio characterizes efficiency and stability of a strategy. It reflects the ratio of the arithmetical mean profit for the position holding time to the standard deviation from it. Plus to that, it accounts for the risk-free rate which is the profit on a certain deposited amount of money.

LR Correlation: linear regression correlation. The balance graph is a broken line, which can be approximated by a straight line for descriptive reasons. To find the coordinates of this straight, the least-squares method is applied. The obtained straight is named "linear regression" and allows one to estimate the balance chart points deviations from the linear regression. Correlation between the balance chart and the linear regression allows estimating the degree of the capital variability. The less sharp peaks and troughs are on the balance curve, the closer to the figure of one is this parameter value. The closer to zero it is, the more random is the trading.

LR Standard Error: the standard error of balance deviation from the linear regression. This index serves to estimate the balance chart deviation from the linear regression in money terms. It is reasonable to compare only systems having similar initial conditions (the same values of the initial equity).The larger is this value, the stronger the balance deviates from the straight.

Margin Level: minimal level of margin in percentage terms registered during testing;

Z-Score: series testing (the probability of correlation between trades). The series testing serves to estimate the degree of correlation between trades and allows one to figure out whether the trade history includes more/less periods of consecutive profits/losses than normal distribution implies. The correlation detected allows one to apply the methods of money management and/or change the trading system algorithm to maximize profit and/or to remove the dependence. Both non-finding the real correlation and finding a nonexistent correlation between trades are dangerous.

OnTester Result: value returned by the OnTester function in the Expert Advisor as a result of testing. Corresponds to custom criterion of optimization.

Total Trades: the total number of trades (deals resulted in fixing a profit/loss).

Total Deals: the total number of deals.

Short Trades (won %): number of deals that have fixed profit from selling a financial instrument and percentage of winning short trades.

Long Trades (won %): number of deals that have fixed profit from buying a financial instrument and percentage of winning long trades.

Profit Trades (% of total): number of winning trades and their percentage in the total number of trades.

Loss Trades (% of total): number of losing trades and their percentage in the total number of trades.

Largest profit trade: the largest profit among all winning trades.

Largest loss trade: the largest loss among all losing trades.

Average profit trade: the average profit value per a trade (the total of profits divided by the number of winning trades).

Average loss trade: the average loss value per a trade (the total of losses divided by the number of losing trades).

Maximum consecutive wins ($): the longest series of winning trades and their total profit.

Maximum consecutive losses ($): the longest series of losing trades and their total loss.

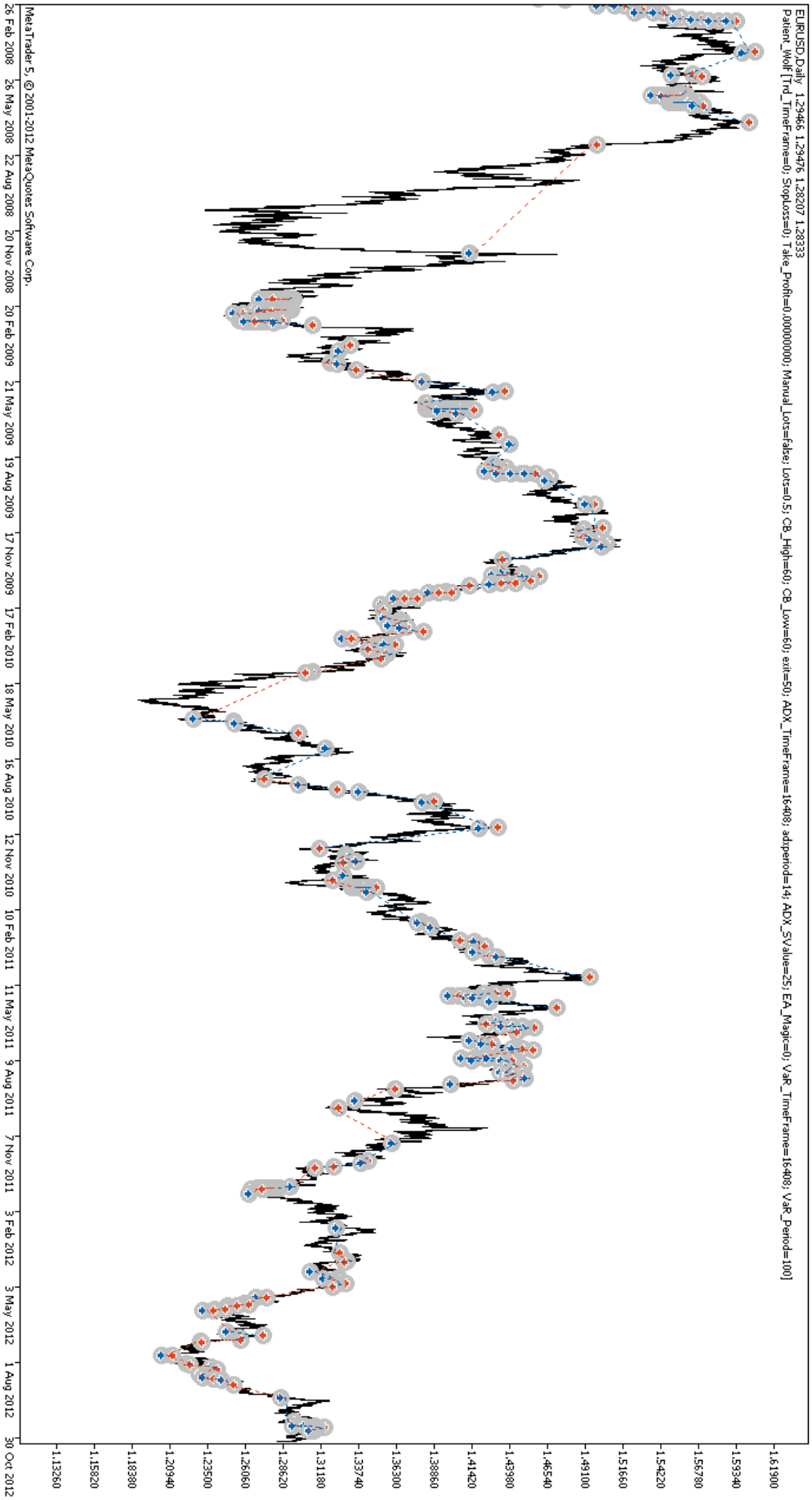
Maximal consecutive profit (count): the maximal profit for one series of winning trades and the number of winning trades in it.

Maximal consecutive loss (count): the maximal loss for one series of losing trades and the number of losing trades in it.

Average consecutive wins: the average number of winning trades in winning series.

Average consecutive losses: the average number of losing trades in losing series.

Figure 4.6.c: *EURUSD Preliminary Test's Trades*



B. Finding the Best Parameters

After conducting our preliminary tests using a set of arbitrary parameters, we will concentrate on finding the best generalized parameters. Since the Metatrader 5 platform contains an optimization module, and since we look for generalizing parameters rather than optimizing them, we are going to use the optimization module but with large periods, so that the resulting optimized parameters could also be considered as general parameters.

Figure 4.7: *Optimization Settings*

Variable	Value	Start	Step	Stop
☐ Trading Time Frame	current	current		current
☐ CB SL, 0 in case of none	0	0	0.0	0.0
☐ CB TP, 0 in case of none	0.0	0.0	0.0	0.0
☐ Manual_Lots	false	false		true
☐ Trade lots in case Manual_Lots is True	1	1.0	0.1	10.0
☐ Prots	0.02	0.02	0.002	0.2
☐ MA_TimeFrame	current	current		1 Month
☐ Period_MA_1	90	90	1	900
☐ Period_MA_2	150	150	1	1500
☑ **CB_High**	**20**	**20**	**5**	**65**
☑ **CB_Low**	**20**	**20**	**5**	**65**
☑ **exit**	**20**	**20**	**5**	**60**
☐ ADX_TimeFrame	current	current		current
☐ adxperiod	12	12	1	120
☑ **ADX critical Value that indicates if the market is trending**	**22.0**	**20**	**4**	**35**
☐ Expert Magic number	0	0	1	10
☐ VaR_TimeFrame	current	current		1 Month
☐ VaR_Period	100	100	1	1000
☐ MA_Method	Simple	Simple		Linear weighted

Figure 4.7 shows the optimization test settings. The optimized parameters are those for the channel breakout strategy, and the ADX critical value for which the market is considered trending. The CB_High is the range for which the highest high of the channel breakout is computed. This parameter will be tested with values ranging from 20 days to 65 days, and increases 5 days with each iteration.

The CB_Low is the range for which the lowest low of the channel breakout is computed. Similar to CB_High, this parameter will be tested with values ranging from 20 days to 65 days and increases 5 days with each iteration.

The ADX critical value is set to start from 20 and increases by 4 with each iteration, till it reaches 35.

The optimization test will continuously iterate each parameter's value until it reaches the best combination that gives maximum profit with the minimal drawdown. The time period used for the optimization test is from 01/01/2010 till 10/23/2012.

The test results are shown in table 4.2. Compared to other parameters, the ones that had the drawdown of 39.09 are considered the best, because they made approximately the same profit but with less risk. For this particular reason we are going to choose these mentioned parameters.

Table 4.2: *Parameters' Optimization Results*

Profit	Profit Factor	Sharpe Ratio	Equity DD %	Trades	CB_High	CB_Low	exit	ADX_SValue
21219.40	1.25	0.12	47.99	135	20	55	20	24
21049.38	1.24	0.12	39.09	128	60	60	50	14
18649.20	1.22	0.12	47.37	129	35	60	25	24
17905.10	1.22	0.12	45.40	123	20	55	40	24
18125.50	1.21	0.12	46.78	129	25	60	30	24
17178.90	1.21	0.12	58.72	122	30	45	55	24
17178.90	1.21	0.12	58.72	122	40	25	55	24
17178.90	1.21	0.12	58.72	122	20	20	55	24
17178.90	1.21	0.12	58.72	122	20	35	60	24
17178.90	1.21	0.12	58.72	122	20	35	55	24
16682.00	1.20	0.12	48.48	124	20	55	35	24
16409.20	1.19	0.11	45.26	128	55	60	25	24
15702.70	1.18	0.11	48.37	123	30	65	45	24
15352.10	1.18	0.11	58.72	124	20	20	40	24
14932.00	1.18	0.11	62.87	121	65	60	55	24
14506.60	1.17	0.11	47.57	128	50	55	20	24
14346.60	1.16	0.11	58.06	128	35	50	30	24
14129.00	1.16	0.11	58.72	125	35	45	35	24
13649.30	1.16	0.11	49.91	128	55	65	20	24
13149.70	1.15	0.11	58.72	124	40	40	50	24
13149.70	1.15	0.11	58.72	124	20	35	45	24
13127.20	1.15	0.10	64.39	128	60	40	20	24
12904.00	1.15	0.11	55.13	126	20	20	35	24
12946.30	1.15	0.10	52.08	128	45	50	30	24
11841.40	1.13	0.10	53.59	123	50	60	50	24
12242.40	1.13	0.10	59.14	141	20	20	20	24
10889.10	1.12	0.10	69.23	132	65	20	25	24
10495.00	1.12	0.10	71.17	123	60	30	45	24
10495.00	1.12	0.10	71.17	123	60	25	45	24
9433.90	1.10	0.10	67.41	128	50	40	30	24
9288.40	1.10	0.10	68.12	124	50	50	50	24
7531.10	1.10	0.13	90.75	83	20	25	35	20
7134.00	1.08	0.09	64.20	126	55	20	35	24
4852.30	1.07	0.12	90.75	82	40	25	35	20
4780.20	1.06	0.10	97.54	100	20	20	20	20
4672.30	1.05	0.08	73.14	134	55	20	20	24
3957.30	1.05	0.10	92.54	94	30	25	20	20
3478.00	1.04	0.12	92.75	82	35	50	20	20
2573.70	1.03	0.13	94.85	83	40	40	30	20
1743.20	1.02	0.12	88.88	80	55	45	20	20
360.10	1.00	0.12	97.34	74	40	55	40	20
-9599.70	0.86	-0.02	100.05	95	20	20	45	32
-9949.80	0.86	-0.02	100.40	94	20	55	45	32

C. The Final Back-test

After choosing the systems' parameters, we are going to test them on the last recent short-term period. The test will be conducted from 01/01/2011 till 10/23/2012, and the results are shown in Figures 4.8.a, 4.8.b, 4.8.c and 4.8.d.

After checking the resulting stats which are self-explanatory, it's obvious that the developed system has succeeded to achieve a total net profit of 11,899 USD with an equity maximal drawdown of 39.09% in a period ranging from 01/01/2011 till 10/23/2012.

Figure 4.8.a: *Final Back-test Stats*

History Quality	100%				
Bars	471	Ticks	37344823	Symbols	1
Initial Deposit	10 000.00				
Total Net Profit	11 899.20	Balance Drawdown Absolute	3 840.60	Equity Drawdown Absolute	5 154.60
Gross Profit	62 385.70	Balance Drawdown Maximal	11 363.50 (37.50%)	Equity Drawdown Maximal	12 012.80 (39.09%)
Gross Loss	-50 486.50	Balance Drawdown Relative	38.41% (3 840.60)	Equity Drawdown Relative	57.77% (6 627.90)
Profit Factor	1.24	Expected Payoff	108.17	Margin Level	338.13%
Recovery Factor	0.99	Sharpe Ratio	0.11	Z-Score	-0.78 (56.46%)
AHPR	1.0135 (1.35%)	LR Correlation	0.83	OnTester result	0
GHPR	1.0072 (0.72%)	LR Standard Error	4 392.50		
Total Trades	110	Short Trades (won %)	52 (55.77%)	Long Trades (won %)	58 (65.52%)
Total Deals	220	Profit Trades (% of total)	67 (60.91%)	Loss Trades (% of total)	43 (39.09%)
	Largest	profit trade	5 645.90	loss trade	-4 366.00
	Average	profit trade	931.13	loss trade	-1 174.10
	Maximum	consecutive wins ($)	11 (16 160.40)	consecutive losses ($)	4 (-9 450.80)
	Maximal	consecutive profit (count)	16 160.40 (11)	consecutive loss (count)	-9 450.80 (4)
	Average	consecutive wins	3	consecutive losses	2

Figure 4.8.b: *Weekly and Monthly P&L*

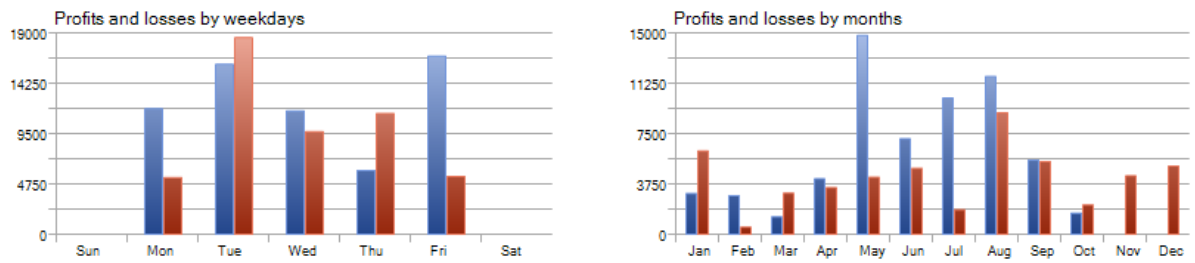


Figure 4.8.c: *Back-test Trade Chart*

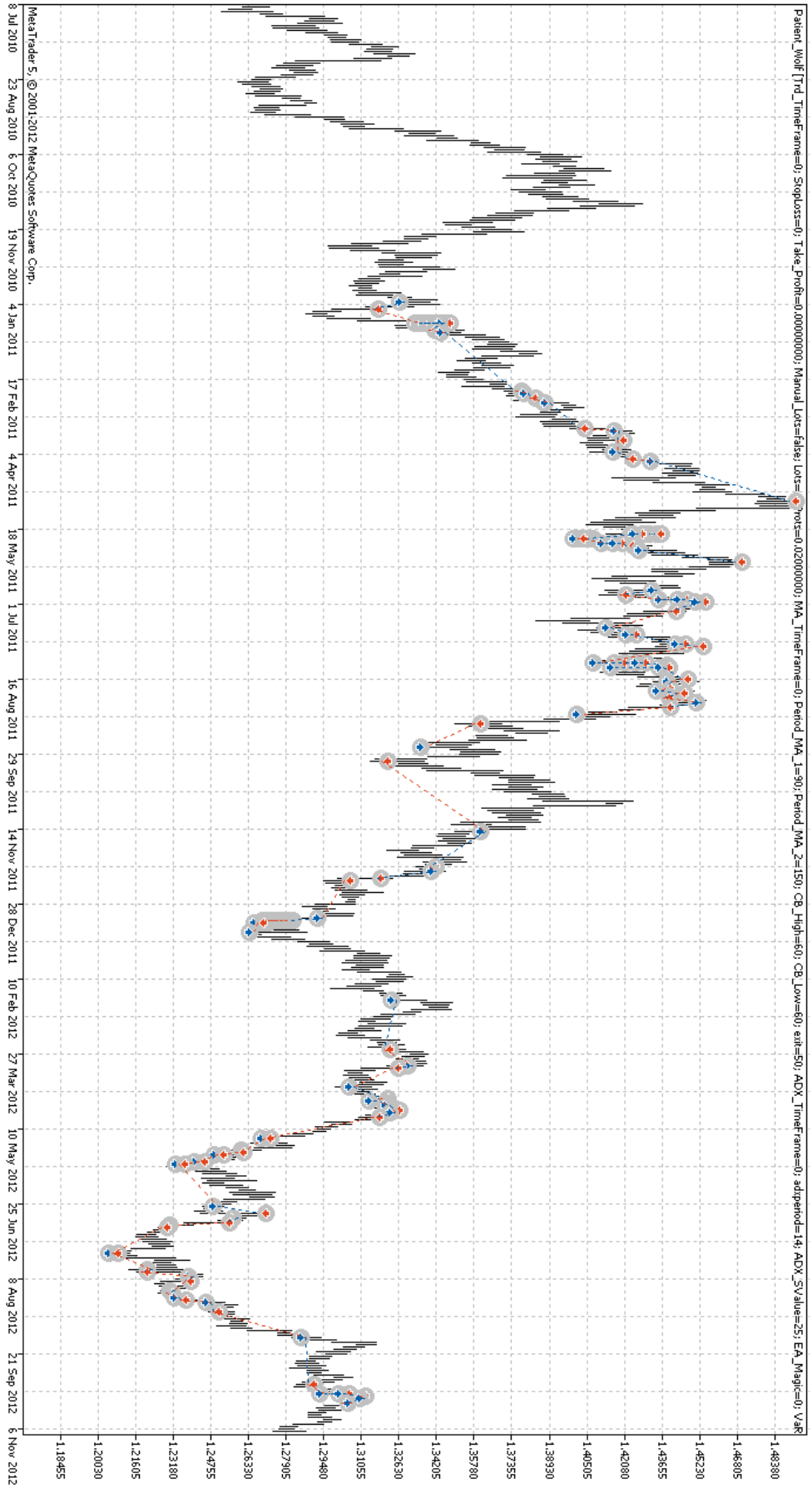
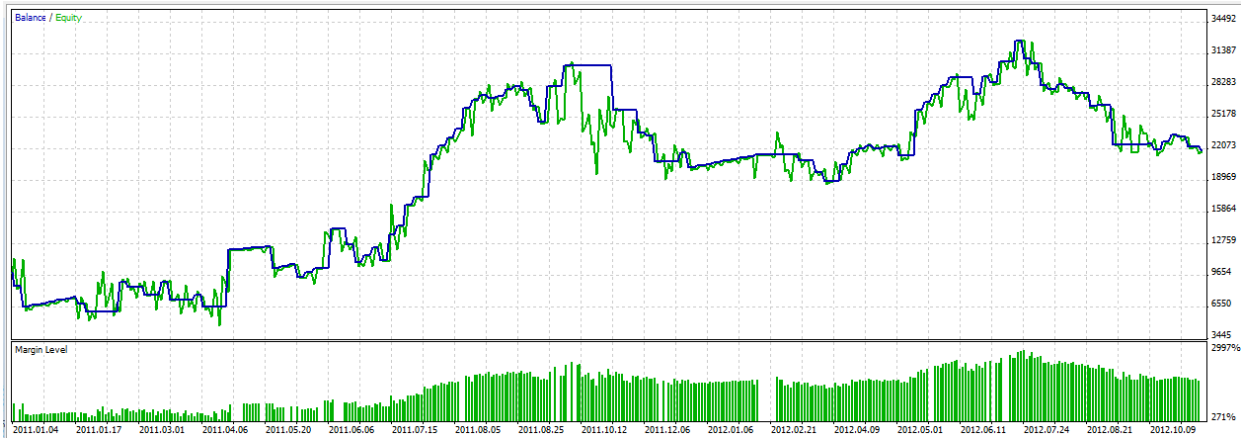


Figure 4.8.d: *Final Back-test's Equity and Balance Curve*



Running the System

After analyzing, planning, developing, implementing and back-testing our algorithmic trading system, we should set it to run on a probationary period with a small amount of money. The probationary period helps evaluating the system in real time, discovering the programming flows and run-time errors. If the system passes the probationary period successfully and if the investors and traders' expectations are met, then the system should be set to run normally.

It's not enough to set the algorithmic trading system to run by its own, it should be kept under continuous supervision and maintenance. The system should always be back-tested to see if the used parameters are still valid or if they should be updated.

Conclusion

Many investors or traders do not believe in algorithmic trading systems. This judgment is due to the lack of knowledge in algorithmic trading and its applications, and to the impression that such systems are not flexible and they forbid external intervention in the training decision.

This study gives a clear idea about Algorithmic Trading, and how the investors and traders' needs can be converted into customized flexible algorithms that can do approximately all they plan and envision. Furthermore, this study describes in details the process of developing, implementing and back-testing algorithmic trading systems. To unleash the potentials of algorithmic trading systems, an algorithm that combines artificial neural network with a channel breakout strategy has been developed. The resulting trading system has been back-tested and found profitable.

As a conclusion, we can assure that algorithmic trading systems' profitability is strictly linked to the investors or traders' ability to envision smart and profitable strategies.

Bibliography

1. **HEATON, Jeff.** *Heaton Research.* [Online] 2012. http://www.heatonresearch.com.

2. **VAN VLIET, Benjamin.** *Building Automated Trading Systems: With an Introduction to Visual C++.NET 2005.* California : Elsevier, 2007. p. 331.

3. **JOHNSON, Barry.** *Algorithmic Trading & DMA : An introduction to direct access trading strategies.* London : 4Myeloma Press, 2010. p. 595.

4. **READ, Colin.** History of Algorithmic Trading Shows Promise and Perils - Bloomberg. *http://www.bloomberg.com.* [Online] August 8, 2012. [Cited: July 24, 2012.] http://www.bloomberg.com/news/2012-08-08/history-of-algorithmic-trading-shows-promise-and-perils.html.

5. **KISSELL, Robert and MALAMUT, Roberto.** Understanding the Profit and Loss Distribution of Trading Algorithms. *http://ts.jpmorgan.com.* [Online] February 2005. http://ts.jpmorgan.com/cm/BlobServer?blobtable=Document&blobcol=urlblob&blobkey=name&blobheader=application/pdf&blobwhere=GES_Understand_PL.pdf.

6. **COVEL, Michael W.** *TREND COMMANDMENTS: TRADING FOR EXCEPTIONAL RETURNS.* New Jersey : Pearson Education, 2011. p. 32.

7. **TSAY, Ruey S.** *Analysis of Financial Time Series.* 2nd. New Jersey : Wiley Interscience, 2005. p. 638.